

ASSIGNMENT – 2

IBM GENERATIVE AI ADVANCED

SUBMITTED BY

NAME: RAJAVARSHINI C

REGISTRATION NUMBER: 23MIC0142

DEPARTMENT: INTEGRATED M.TECH - COMPUTER SCIENCE AND
ENGINEERING

COLLEGE NAME: VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

COURSE NAME: IBM GENERATIVE AI ADVANCED

BATCH NAME: GENAI-03

SLOT: 2

SUBMITTED TO: MR. MASIL GOKUL SELVEN S

DATE OF SUBMISSION: JUNE 30, 2026

MINI PROJECT – 1

DATA SCIENCE ASSIGNMENT

PART 1 – DATASET SELECTION

Name of the Dataset: Boston Housing Dataset

Source: [Boston Housing Dataset with column names](#)

Number of rows and columns: 506 rows and 14 columns

Target variable: MEDV

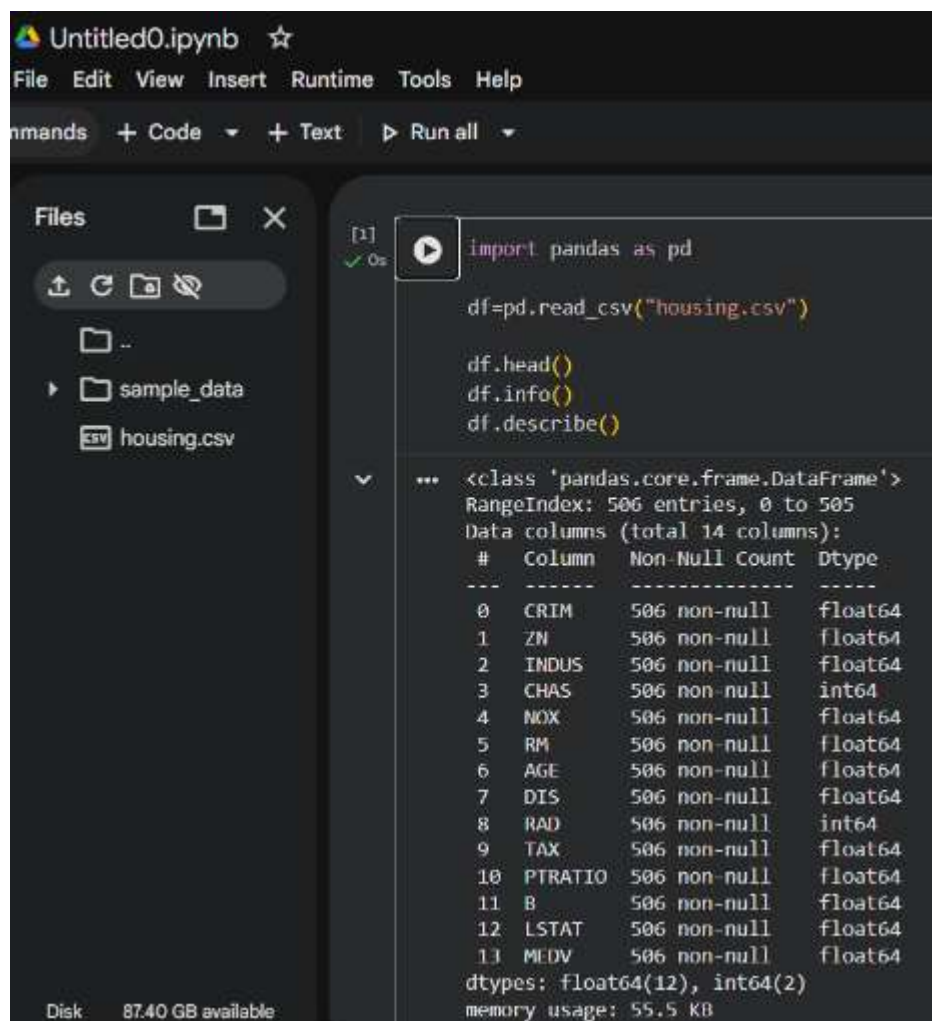
The reason why I choose this dataset: I selected the Boston Housing dataset because it is widely used for regression problems. It contains several housing-related features that influence house prices. The dataset is suitable for learning data preprocessing, visualization, and predictive modelling.

Prompt to ChatGPT:

"I am using the Boston Housing dataset."

Generate Python code to load the CSV file using pandas and display df.head(), df.info(), df.describe(), shape, and missing values."

Output:



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes the file name 'Untitled0.ipynb' and a star icon. Below the bar are tabs for 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A toolbar shows '+ Code', '+ Text', and 'Run all'. The left sidebar displays a file explorer with a folder 'sample_data' containing a file 'housing.csv'. The main area shows a code cell with the following Python code:

```
[1] import pandas as pd
df=pd.read_csv("housing.csv")

df.head()
df.info()
df.describe()
```

The output of the code cell is displayed below the code:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0    CRIM        506 non-null    float64
1    ZN          506 non-null    float64
2    INDUS       506 non-null    float64
3    CHAS        506 non-null    int64
4    NOX         506 non-null    float64
5    RM          506 non-null    float64
6    AGE         506 non-null    float64
7    DIS         506 non-null    float64
8    RAD         506 non-null    int64
9    TAX         506 non-null    float64
10   PTRATIO     506 non-null    float64
11   B           506 non-null    float64
12   LSTAT       506 non-null    float64
13   MEDV        506 non-null    float64
dtypes: float64(12), int64(2)
memory usage: 55.5 KB
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901	3.795043	9.549407	408.237154	18.455534	356.674032	12.653063	22.532806
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861	2.105710	8.707259	168.537115	2.164945	51.294854	7.141062	9.197104
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000	1.129600	1.000000	187.000000	12.600000	0.320000	1.730000	5.000000
25%	0.062045	0.000000	5.190000	0.000000	0.448000	5.885500	45.025000	2.100175	4.000000	279.000000	17.400000	375.377500	6.950000	17.025000
50%	0.256510	0.000000	8.690000	0.000000	0.538000	6.208500	77.500000	3.207450	5.000000	330.000000	19.050000	391.440000	11.360000	21.200000
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.823500	94.075000	5.188425	24.000000	686.000000	20.200000	396.225000	16.955000	25.000000
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000	12.126500	24.000000	711.000000	22.000000	396.900000	37.970000	50.000000

PART 2 – ETL PIPELINE

Prompt:

"I have a dataset called boston housing dataset with 14 columns

Write a Python ETL pipeline that:

1. Loads the raw CSV from a raw/ folder
2. Drops duplicate rows
3. Handles missing values appropriately
4. Converts data types where needed
5. Saves the clean data to a gold/ folder as clean_data.csv"

Housing(1)

	CRIM	ZN	INDUS	CHAS
1	0.00632	18.0	2.31	0
2	0.02731	0.0	7.07	0
3	0.02729	0.0	7.07	0
4	0.032369999999999996	0.0	2.18	0
5	0.06905	0.0	2.18	0

"I have a dataset called boston housing dataset with 14 columns
Write a Python ETL pipeline that:

1. Loads the raw CSV from a raw/ folder
2. Drops duplicate rows
3. Handles missing values appropriately
4. Converts data types where needed
5. Saves the clean data to a gold/ folder as clean_data.csv"

Here's a complete Python ETL pipeline that satisfies all the requirements. It assumes your dataset is stored in a folder named `raw/` and saves the cleaned dataset to `gold/clean_data.csv`.

```

Python
import os
import pandas as pd

# -----
# ETL Pipeline For Boston Housing Dataset
  
```

Run

CODE

```
import pandas as pd
import os

os.makedirs("raw", exist_ok=True)
os.makedirs("gold", exist_ok=True)

print("Folders created successfully!")
```

Folders created successfully!

```
from google.colab import files

uploaded = files.upload()
```

Choose files housing.csv

housing.csv(text/csv) - 38955 bytes, last modified: 26/06/2026 - 100% done
Saving housing.csv to housing.csv

```
import shutil

shutil.move("housing.csv", "raw/housing.csv")

print("File moved to raw folder.")
```

File moved to raw folder.

```
df = pd.read_csv("raw/housing.csv")
```

```
print(df.head())
print("Shape:", df.shape)
```

```
...      CRIM    ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  \
0  0.00632  18.0    2.31    0    0.538  6.575  65.2  4.0900    1  296.0
1  0.02731   0.0    7.07    0    0.469  6.421  78.9  4.9671    2  242.0
2  0.02729   0.0    7.07    0    0.469  7.185  61.1  4.9671    2  242.0
3  0.03237   0.0    2.18    0    0.458  6.998  45.8  6.0622    3  222.0
4  0.06905   0.0    2.18    0    0.458  7.147  54.2  6.0622    3  222.0

      PTRATIO      B  LSTAT  MEDV
0      15.3  396.90   4.98  24.0
1      17.8  396.90   9.14  21.6
2      17.8  392.83   4.03  34.7
3      18.7  394.63   2.94  33.4
4      18.7  396.90   5.33  36.2
Shape: (506, 14)
```

```
df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)
```

Shape after removing duplicates: (506, 14)

```

# Fill missing values in numeric columns with the median
numeric_cols = df.select_dtypes(include=['number']).columns

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

# Fill missing values in categorical columns with the mode
categorical_cols = df.select_dtypes(include=['object']).columns

for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

print("Missing values handled.")

... Missing values handled.

```

```

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

print(df.dtypes)

```

CRIM	float64
ZN	float64
INDUS	float64
CHAS	int64
NOX	float64
RM	float64
AGE	float64
DIS	float64
RAD	int64
TAX	float64
PTRATIO	float64
B	float64
LSTAT	float64
MEDV	float64
dtype:	object

```
df.to_csv("gold/clean_data.csv", index=False)
```

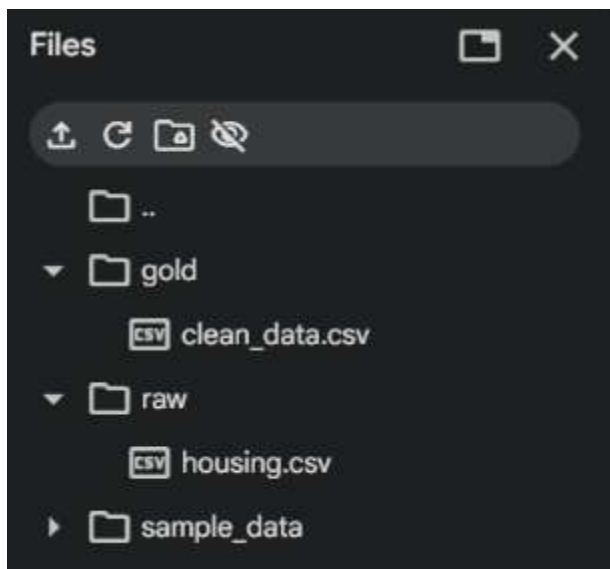
```
print("Cleaned data saved successfully!")
```

```
Cleaned data saved successfully!
```

```
print(os.listdir("gold"))
```

```
['clean_data.csv']
```

FOLDER STRUCTURE



clean_data.csv

1 to 10 of 506 entries Filter

CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0.00632	18.0	2.31	0	0.538	6.575	65.2	4.09	1	296.0	15.3	396.9	4.98	24.0
0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.9	9.14	21.6
0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
0.03236999999999999	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.9	5.33	36.2
0.02985	0.0	2.18	0	0.458	6.43	58.7	6.0622	3	222.0	18.7	394.12	5.21	28.7
0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311.0	15.2	395.6	12.43	22.9
0.14455	12.5	7.87	0	0.524	6.172	96.1	5.9505	5	311.0	15.2	396.9	19.15	27.1
0.21124	12.5	7.87	0	0.524	5.631	100.0	6.0821	5	311.0	15.2	386.63	29.93	16.5
0.17004	12.5	7.87	0	0.524	6.004	85.9	6.5921	5	311.0	15.2	386.71	17.1	18.9

Show 10 per page

1 2 10 50 51

PART 3 – DATABASE SCHEMA

Prompt:

"I am using the Boston Housing dataset with the following columns:

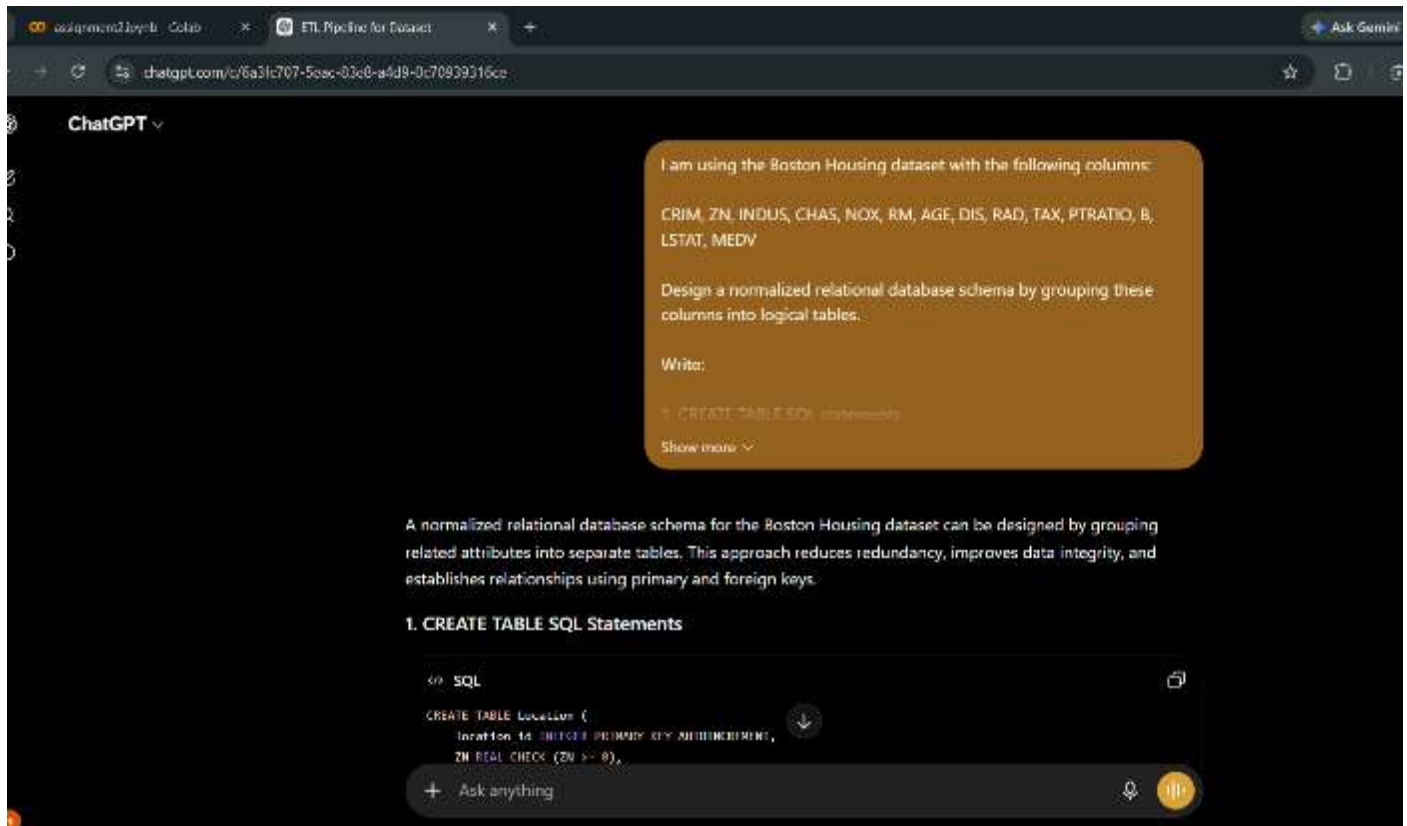
CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT, MEDV

Design a normalized relational database schema by grouping these columns into logical tables.

Write:

1. CREATE TABLE SQL statements

2. Primary Keys
3. Foreign Keys
4. At least three CHECK constraints
5. ERD description
6. Python SQLite code to create the database and insert the cleaned data.”



TABLES

Table1 – Property

property_id
CRIM
ZN
INDUS
CHAS

Table2 – Environment

env_id
property_id
NOX
AGE
DIS

Table3 – Infrastructure

infra_id
property_id
RAD
TAX
PTRATIO

Table4 – Housing

housing_id
property_id
RM
B
MEDV

A normalized relational database schema for the Boston Housing dataset can be designed by grouping related attributes into separate tables. This approach reduces redundancy, improves data integrity, and establishes relationships using primary and foreign keys.

1. CREATE TABLE SQL Statements

 SQL 

```
CREATE TABLE Location (  
  location_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  ZN REAL CHECK (ZN >= 0),  
  INDUS REAL CHECK (INDUS >= 0),  
  CHAS INTEGER CHECK (CHAS IN (0,1))  
);  
  
CREATE TABLE Environment (  
  environment_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  NOX REAL CHECK (NOX > 0),  
  AGE REAL CHECK (AGE >= 0),  
  DIS REAL CHECK (DIS >= 0)  
);  
  
CREATE TABLE Accessibility (  
  access_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  RAD INTEGER CHECK (RAD >= 1),  
  TAX INTEGER CHECK (TAX >= 0),  
  PTRATIO REAL CHECK (PTRATIO > 0)  
);  
  
CREATE TABLE Housing (  
  house_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  CRIM REAL CHECK (CRIM >= 0),  
  RM REAL CHECK (RM > 0),  
  B REAL CHECK (B >= 0),  
  LSTAT REAL CHECK (LSTAT >= 0),  
  MEDV REAL CHECK (MEDV >= 0),  
  
  location_id INTEGER,  
  environment_id INTEGER,
```



```
  access_id INTEGER,  
  
  FOREIGN KEY (location_id) REFERENCES Location(location_id),  
  FOREIGN KEY (environment_id) REFERENCES Environment(environment_id),  
  FOREIGN KEY (access_id) REFERENCES Accessibility(access_id)  
);
```

2.PRIMARY KEYS

Table	Primary Key
Location	location_id
Environment	environment_id
Accessibility	access_id
Housing	house_id

3.FOREIGN KEYS

Table	Foreign Key	References
Housing	location_id	Location(location_id)
Housing	environment_id	Environment(environment_id)
Housing	access_id	Accessibility(access_id)

4. CHECK CONSTRAINTS

CHECK (CHAS IN (0,1))

CHECK (CRIM >= 0)

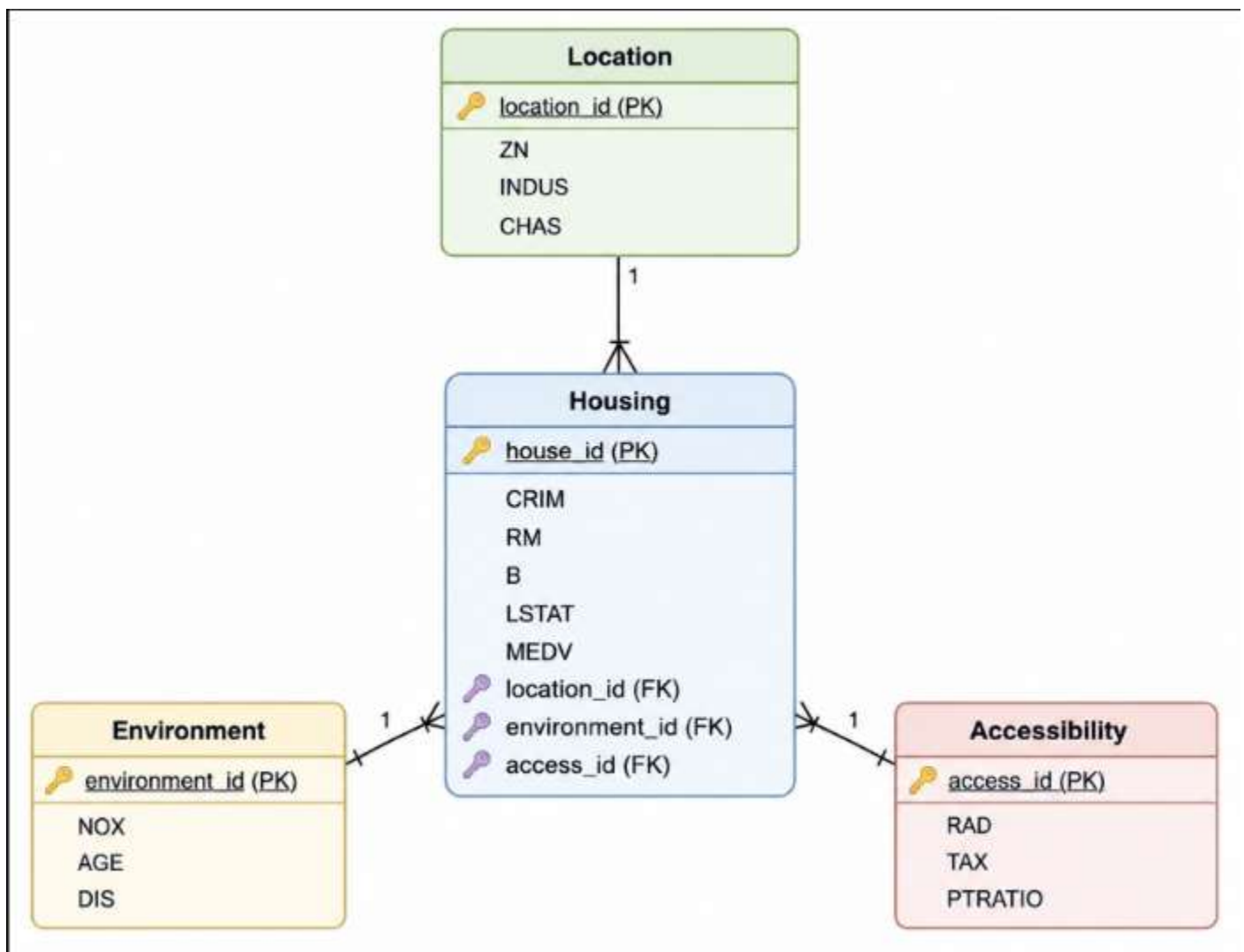
CHECK (RM > 0)

CHECK (NOX > 0)

CHECK (MEDV >= 0)

CHECK (PTRATIO > 0)

5. ERD DESCRIPTION



6. PYTHON SQLite CODE

```
import sqlite3
import pandas as pd

# Load the cleaned dataset
df = pd.read_csv("gold/clean_data.csv")

# Create SQLite database
conn = sqlite3.connect("boston_housing.db")
cursor = conn.cursor()

# Enable foreign key support
cursor.execute("PRAGMA foreign_keys = ON")

# Create Location table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Location (
    location_id INTEGER PRIMARY KEY AUTOINCREMENT,
    ZN REAL CHECK (ZN >= 0),
    INDUS REAL CHECK (INDUS >= 0),
    CHAS INTEGER CHECK (CHAS IN (0,1))
)
""")
```

```
# Create Environment table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Environment (
    environment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    NOX REAL CHECK (NOX > 0),
    AGE REAL CHECK (AGE >= 0),
    DIS REAL CHECK (DIS >= 0)
)
""")

# Create Accessibility table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Accessibility (
    access_id INTEGER PRIMARY KEY AUTOINCREMENT,
    RAD INTEGER CHECK (RAD >= 1),
    TAX INTEGER CHECK (TAX >= 0),
    PTRATIO REAL CHECK (PTRATIO > 0)
)
""")

# Create Housing table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Housing (
    house_id INTEGER PRIMARY KEY AUTOINCREMENT,
    CRIM REAL CHECK (CRIM >= 0),
    RM REAL CHECK (RM > 0),
    B REAL CHECK (B >= 0),
    LSTAT REAL CHECK (LSTAT >= 0),
    MEDV REAL CHECK (MEDV >= 0),

    location_id INTEGER,
    environment_id INTEGER,
    access_id INTEGER,

    FOREIGN KEY(location_id) REFERENCES Location(location_id),
    FOREIGN KEY(environment_id) REFERENCES Environment(environment_id),
    FOREIGN KEY(access_id) REFERENCES Accessibility(access_id)
)
""")
```

```

▶ # Insert data into tables
for _, row in df.iterrows():

    cursor.execute("""
        INSERT INTO Location (ZN, INDUS, CHAS)
        VALUES (?, ?, ?)
    """, (row["ZN"], row["INDUS"], row["CHAS"]))
    location_id = cursor.lastrowid

    cursor.execute("""
        INSERT INTO Environment (NOX, AGE, DIS)
        VALUES (?, ?, ?)
    """, (row["NOX"], row["AGE"], row["DIS"]))
    environment_id = cursor.lastrowid

    cursor.execute("""
        INSERT INTO Accessibility (RAD, TAX, PTRATIO)
        VALUES (?, ?, ?)
    """, (row["RAD"], row["TAX"], row["PTRATIO"]))
    access_id = cursor.lastrowid

    cursor.execute("""
        INSERT INTO Housing (
            CRIM, RM, B, LSTAT, MEDV,
            location_id,
            environment_id,
            access_id
        )
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        row["CRIM"],
        row["RM"],
        row["B"],
        row["LSTAT"],
        row["MEDV"],
        location_id,
        environment_id,
        access_id
    ))

conn.commit()

```

```

print("Database created successfully.")
print("Data inserted successfully.")

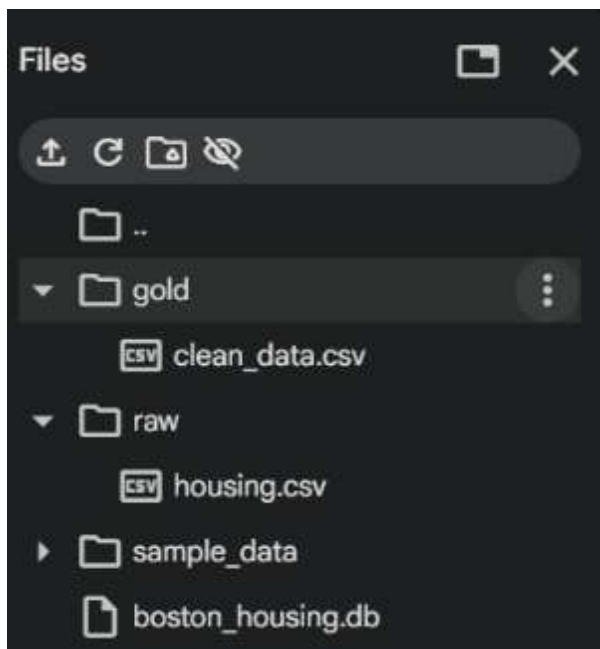
conn.close()

```

```

*** Database created successfully.
    Data inserted successfully.

```



```
boston_housing.db

1 SQLite format 3
2
3
4 house_id INTEGER PRIMARY KEY AUTOINCREMENT,
5 CRIM REAL CHECK (CRIM >= 0),
6 RM REAL CHECK (RM > 0),
7 B REAL CHECK (B >= 0),
8 LSTAT REAL CHECK (LSTAT >= 0),
9 MEDV REAL CHECK (MEDV >= 0),
10
11 location_id INTEGER,
12 environment_id INTEGER,
13 access_id INTEGER,
14
15 FOREIGN KEY(location_id) REFERENCES Location(location_id),
16 FOREIGN KEY(environment_id) REFERENCES Environment(environment_id),
17 FOREIGN KEY(access_id) REFERENCES Accessibility(access_id)
18 )
19 access_id INTEGER PRIMARY KEY AUTOINCREMENT,
20 RAD INTEGER CHECK (RAD >= 1),
21 TAX INTEGER CHECK (TAX >= 0),
22 PTRATIO REAL CHECK (PTRATIO > 0)
23 )
24 environment_id INTEGER PRIMARY KEY AUTOINCREMENT,
25 NOX REAL CHECK (NOX > 0),
26 AGE REAL CHECK (AGE >= 0),
27 DIS REAL CHECK (DIS >= 0)
28 )
29 sqlite_sequence(name,seq)
30
```

PART 4 – EXPLORATORY DATA ANALYSIS

4A – DESCRIPTIVE STATISTICS

Prompt:

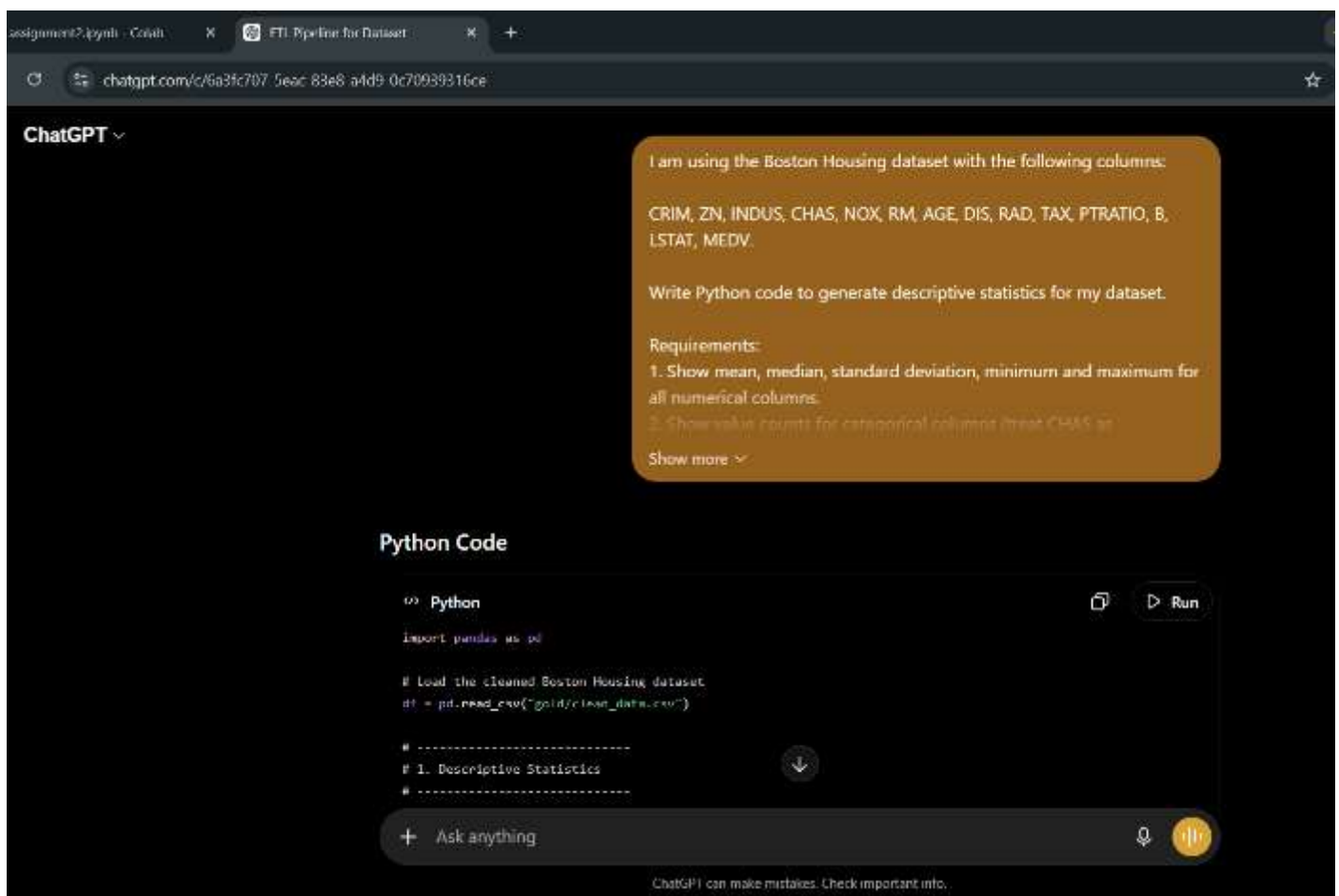
“I am using the Boston Housing dataset with the following columns:

CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT, MEDV.

Write Python code to generate descriptive statistics for my dataset.

Requirements:

- 1. Show mean, median, standard deviation, minimum and maximum for all numerical columns.*
- 2. Show value counts for categorical columns (treat CHAS as categorical).*
- 3. Calculate the count and percentage of missing values for each column.*
- 4. Explain the statistics in plain English.”*



CODE

```
13]  import pandas as pd

# Load cleaned dataset
df = pd.read_csv("gold/clean_data.csv")

# Numerical summary
print("===== DESCRIPTIVE STATISTICS =====")
print(df.describe())

# Mean
print("\nMean")
print(df.mean(numeric_only=True))

# Median
print("\nMedian")
print(df.median(numeric_only=True))

# Standard Deviation
print("\nStandard Deviation")
print(df.std(numeric_only=True))

# Minimum
print("\nMinimum")
print(df.min(numeric_only=True))

# Maximum
print("\nMaximum")
print(df.max(numeric_only=True))

# Treat CHAS as categorical
print("\nCHAS Value Counts")
print(df["CHAS"].value_counts())

# Missing values
print("\nMissing Values")
missing = df.isnull().sum()

missing_percent = (missing/len(df))*100
```

```
missing_table = pd.DataFrame({
    "Missing Count": missing,
    "Percentage": missing_percent
})

print(missing_table)
```

OUTPUT

===== DESCRIPTIVE STATISTICS =====							
***		CRIM	ZN	INDUS	CHAS	NOX	RM \
	count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
	mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634
	std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617
	min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000
	25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500
	50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500
	75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500
	max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000
		AGE	DIS	RAD	TAX	PTRATIO	B \
	count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
	mean	68.574901	3.795043	9.549407	408.237154	18.455534	356.674032
	std	28.148861	2.105710	8.707259	168.537116	2.164946	91.294864
	min	2.900000	1.129600	1.000000	187.000000	12.600000	0.320000
	25%	45.025000	2.100175	4.000000	279.000000	17.400000	375.377500
	50%	77.500000	3.207450	5.000000	330.000000	19.050000	391.440000
	75%	94.075000	5.188425	24.000000	666.000000	20.200000	396.225000
	max	100.000000	12.126500	24.000000	711.000000	22.000000	396.900000
		LSTAT	MEDV				
	count	506.000000	506.000000				
	mean	12.653063	22.532806				
	std	7.141062	9.197104				
	min	1.730000	5.000000				
	25%	6.950000	17.025000				
	50%	11.360000	21.200000				
	75%	16.955000	25.000000				
	max	37.970000	50.000000				

```
*** Mean
    CRIM      3.613524
    ZN        11.363636
    INDUS     11.136779
    CHAS       0.069170
    NOX        0.554695
    RM         6.284634
    AGE        68.574901
    DIS         3.795043
    RAD         9.549407
    TAX        408.237154
    PTRATIO    18.455534
    B          356.674032
    LSTAT      12.653063
    MEDV       22.532806
dtype: float64
```

```
*** Median
    CRIM      0.25651
    ZN         0.00000
    INDUS      9.69000
    CHAS       0.00000
    NOX        0.53800
    RM         6.20850
    AGE        77.50000
    DIS         3.20745
    RAD         5.00000
    TAX        330.00000
    PTRATIO    19.05000
    B          391.44000
    LSTAT      11.36000
    MEDV       21.20000
dtype: float64
```



```
Standard Deviation
*** CRIM      8.601545
    ZN       23.322453
    INDUS    6.860353
    CHAS     0.253994
    NOX      0.115878
    RM       0.702617
    AGE      28.148861
    DIS      2.105710
    RAD      8.707259
    TAX     168.537116
    PTRATIO  2.164946
    B       91.294864
    LSTAT    7.141062
    MEDV     9.197104
dtype: float64
```

```
*** Minimum
    CRIM      0.00632
    ZN        0.00000
    INDUS    0.46000
    CHAS      0.00000
    NOX       0.38500
    RM        3.56100
    AGE       2.90000
    DIS       1.12960
    RAD        1.00000
    TAX     187.00000
    PTRATIO   12.60000
    B         0.32000
    LSTAT     1.73000
    MEDV      5.00000
dtype: float64
```

```
Maximum
    CRIM      88.9762
    ZN       100.0000
    INDUS    27.7400
    CHAS      1.0000
    NOX       0.8710
    RM        8.7800
    AGE     100.0000
    DIS     12.1265
    RAD     24.0000
    TAX     711.0000
    PTRATIO  22.0000
    B       396.9000
    LSTAT   37.9700
    MEDV    50.0000
dtype: float64
```

```
CHAS Value Counts
CHAS
0    471
1     35
Name: count, dtype: int64
```

```
Missing Values
      Missing Count  Percentage
CRIM                0          0.0
ZN                  0          0.0
INDUS               0          0.0
CHAS                0          0.0
NOX                 0          0.0
RM                  0          0.0
AGE                 0          0.0
DIS                 0          0.0
RAD                 0          0.0
TAX                 0          0.0
PTRATIO             0          0.0
B                   0          0.0
LSTAT               0          0.0
MEDV                0          0.0
```

INTERPRETATION

The descriptive statistics provide an overview of the Boston Housing dataset, which contains 506 records. The average number of rooms (RM) is 6.28, indicating that most houses have around six rooms, while the average median house value (MEDV) is 22.53. The CRIM variable has a relatively high standard deviation (8.60), showing that crime rates vary considerably across different neighborhoods. The CHAS value counts indicate that 471 properties do not border the Charles River, while only 35 properties do, making it a highly imbalanced categorical feature. The missing value analysis shows that all columns have 0 missing values (0%), confirming that the dataset has been successfully cleaned and is ready for further exploratory analysis and predictive modeling.

4B – UNIVARIATE ANALYSIS

Prompt:

"I am using the Boston Housing dataset with the following columns:

CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT, MEDV.

Write Python code to perform univariate analysis.

Requirements:

- 1. Create histograms for all numerical columns.*
- 2. Create boxplots to identify outliers.*
- 3. Create bar charts for categorical columns (treat CHAS as categorical).*
- 4. Identify skewed distributions and outliers.*
- 5. Provide observations for each chart."*

assignment2.py:nb - Code Editor Pipeline for Dataset

chatgpt.com/c/1a3fc787-5aac-83e8-a4d9-bc70939316ca

ChatGPT

I am using the Boston Housing dataset with the following columns:

CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT, MEDV.

Write Python code to perform univariate analysis.

Requirements:

1. Create histograms for all numerical columns.
2. Create boxplots to identify outliers.
3. Create bar charts for categorical columns (treat CHAS as categorical).
4. Identify skewed distributions and outliers.
5. Provide observations for each chart.

Show less

Python Code

```
Python
import pandas as pd
import matplotlib.pyplot as plt

# Load the cleaned Boston Housing dataset
df = pd.read_csv("gold/clean_data.csv")

# Treat CHAS as a categorical column
numerical_columns = [col for col in df.columns if col != "CHAS"]
categorical_columns = ["CHAS"]
```

+ Ask anything

ChatGPT can make mistakes. Check important info.

31°C Mostly clear

Search

CODE



```
import pandas as pd
import matplotlib.pyplot as plt

# Load cleaned data
df = pd.read_csv("gold/clean_data.csv")

# -----
# Histograms
# -----

numerical_cols = df.select_dtypes(include=["int64", "float64"]).columns

for col in numerical_cols:
    plt.figure(figsize=(6,4))
    plt.hist(df[col], bins=20, edgecolor="black")
    plt.title(f"Histogram of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.show()

# -----
# Boxplots
# -----

for col in numerical_cols:
    plt.figure(figsize=(6,2))
    plt.boxplot(df[col], vert=False)
    plt.title(f"Boxplot of {col}")
    plt.show()

# -----
# Bar Chart (CHAS)
# -----

plt.figure(figsize=(5,4))
df["CHAS"].value_counts().plot(kind="bar")
plt.title("CHAS Value Counts")
plt.xlabel("CHAS")
plt.ylabel("Count")
plt.show()
```

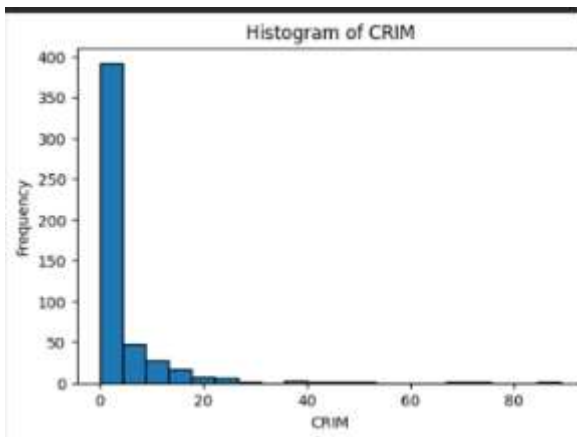


```
# Bar Chart 1
(df["CHAS"].value_counts(normalize=True)*100).plot(kind="bar")
plt.title("CHAS Percentage")
plt.ylabel("Percentage")
plt.show()

# Bar Chart 2
df.groupby("CHAS").size().plot(kind="bar")
plt.title("Properties by CHAS")
plt.show()
```

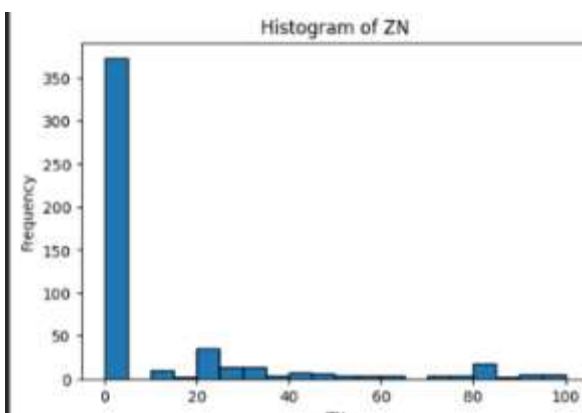
OUTPUT

Histogram of CRIM



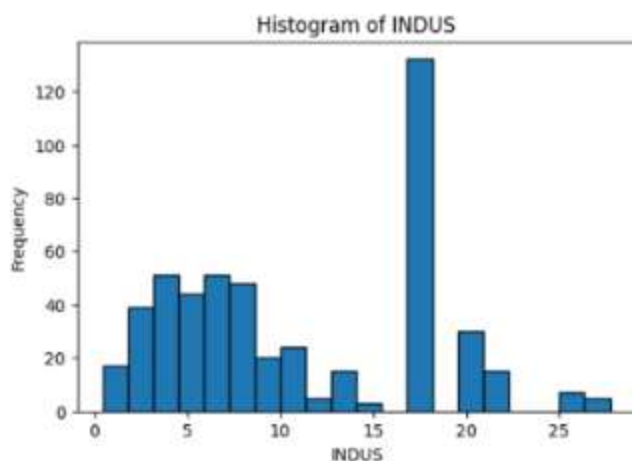
The histogram of CRIM is highly right-skewed. Most properties have low crime rates, while only a few properties have very high crime rates. This indicates the presence of extreme values in the dataset.

Histogram of ZN



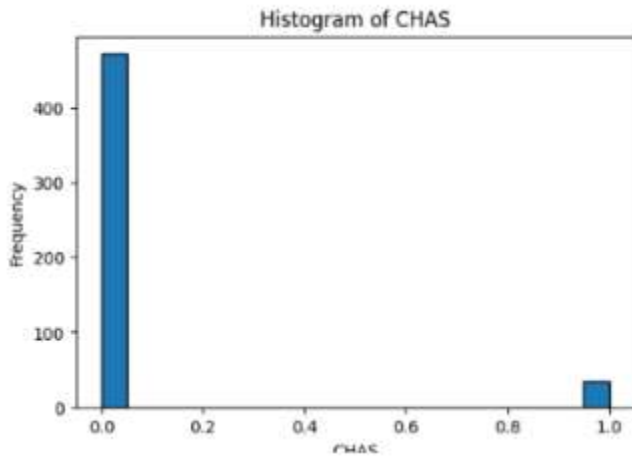
The ZN distribution is positively skewed. Most properties have a zoning value of 0, while only a small number of properties have higher residential zoning percentages.

Histogram of INDUS



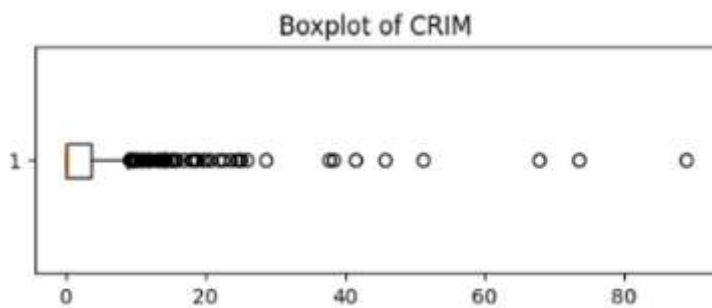
The INDUS variable is not normally distributed and shows multiple peaks. This suggests that properties belong to different industrial area categories.

Histogram of CHAS



The CHAS variable is a binary categorical feature. Most properties have a value of 0 (not bordering the Charles River), while only a few properties have a value of 1, indicating an imbalanced distribution.

Boxplot of CRIM



The boxplot of CRIM shows several outliers above the upper whisker. These outliers represent neighborhoods with unusually high crime rates compared to the majority of the dataset.

Boxplot of ZN



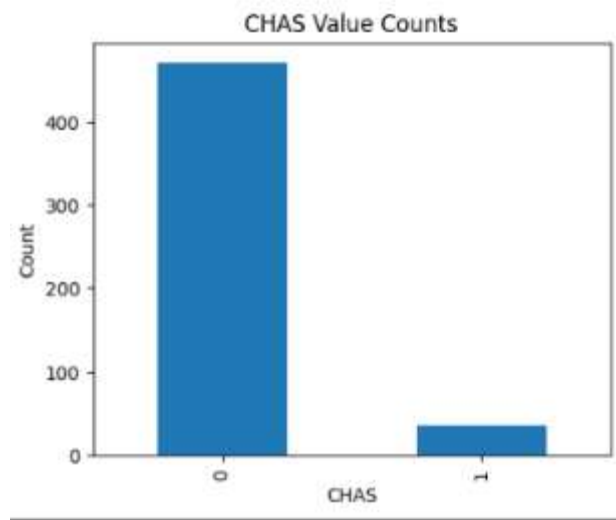
The ZN boxplot contains many outliers because most observations are concentrated at lower values, while only a few properties have very high zoning percentages.

Boxplot of INDUS



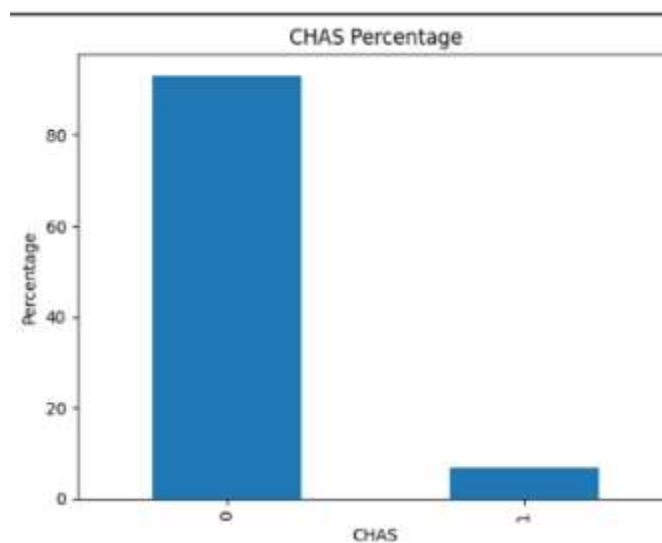
The INDUS boxplot shows very few or no significant outliers. Most observations are spread within a moderate range, indicating relatively consistent industrial land proportions.

CHAS Value Counts



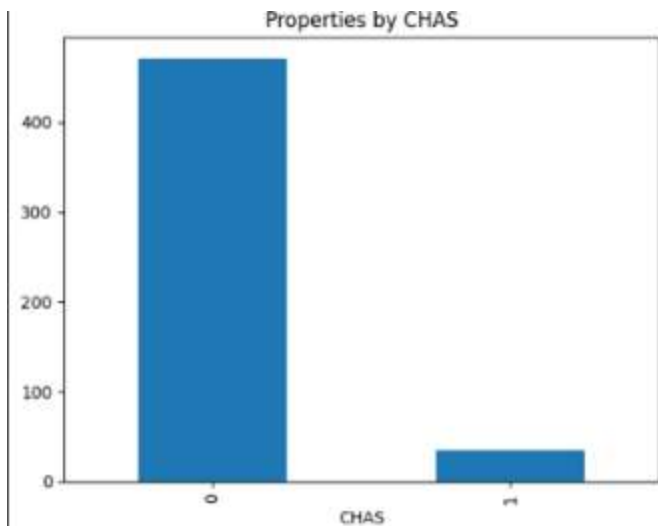
The CHAS bar chart shows that 471 properties do not border the Charles River, whereas only 35 properties do. This indicates that the CHAS feature is highly imbalanced.

CHAS Percentage



Approximately 93% of the properties have CHAS = 0, while only about 7% have CHAS = 1. Therefore, very few houses are located near the Charles River.

Properties by CHAS



The chart confirms that the majority of properties belong to the CHAS = 0 category, while only a small number fall into CHAS = 1. This reinforces the imbalance observed in the categorical distribution.

4C – BIVARIATE AND MULTIVARIATE ANALYSIS

Prompt:

"I am using the Boston Housing dataset with the following columns:

CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT, MEDV.

Write Python code for bivariate and multivariate analysis.

Requirements:

- 1. Create a correlation heatmap of all numerical columns using seaborn.*
- 2. Create scatter plots for the top three features against the target variable MEDV.*
- 3. Create box plots comparing MEDV across categorical groups.*
- 4. Identify the top three strongest predictors of MEDV.*
- 5. Create a combined dashboard with six charts using plt.subplots().*
- 6. Explain the relationships found."*

assignment2.py:nb - Colab ETL Pipeline for Dataset

chatgpt.com/c/6a3fc707-5eac-83e8-a4d9-0c70939316ce

ChatGPT

I am using the Boston Housing dataset with the following columns:
CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT, MEDV.

Write Python code for bivariate and multivariate analysis.

Requirements:
1. Create a correlation heatmap of all numerical columns using seaborn.
2. Create scatter plots for the top three features against the target variable MEDV.

Show more

Python Code

```
Python

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the cleaned Boston Housing dataset
df = pd.read_csv('gold/clean_data.csv')

# -----
# 1. Correlation Heatmap
# -----

plt.figure(figsize=(12,8))
correlation_matrix = df.corr()
```

+ Ask anything

ChatGPT can make mistakes. Check important info.

CODE

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load cleaned data
df = pd.read_csv("gold/clean_data.csv")

# -----
# Correlation Heatmap
# -----

plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

# -----
# Top 3 Features vs Target
# -----

top_features = ["RM", "LSTAT", "PTRATIO"]

for col in top_features:
    plt.figure(figsize=(6,4))
    sns.scatterplot(data=df, x=col, y="MEDV")
    plt.title(f"{col} vs MEDV")
    plt.show()
```

```

# -----
# Boxplots
# -----

# CHAS
plt.figure(figsize=(6,4))
sns.boxplot(x="CHAS", y="MEDV", data=df)
plt.title("CHAS vs MEDV")
plt.show()

# RAD (treated as category)
plt.figure(figsize=(10,4))
sns.boxplot(x="RAD", y="MEDV", data=df)
plt.title("RAD vs MEDV")
plt.show()

# ZN (rounded categories)
df["ZN_group"] = df["ZN"].round()

plt.figure(figsize=(10,4))
sns.boxplot(x="ZN_group", y="MEDV", data=df)
plt.title("ZN vs MEDV")
plt.show()

# -----
# Top 3 Predictors
# -----

corr = df.corr()["MEDV"].abs().sort_values(ascending=False)

print("Top Predictors")
print(corr[1:4])

```

```

# -----
# Dashboard
# -----

fig, axes = plt.subplots(3,2, figsize=(14,14))

sns.scatterplot(data=df,x="RM",y="MEDV",ax=axes[0,0])

sns.scatterplot(data=df,x="LSTAT",y="MEDV",ax=axes[0,1])

sns.scatterplot(data=df,x="PTRATIO",y="MEDV",ax=axes[1,0])

sns.boxplot(data=df,x="CHAS",y="MEDV",ax=axes[1,1])

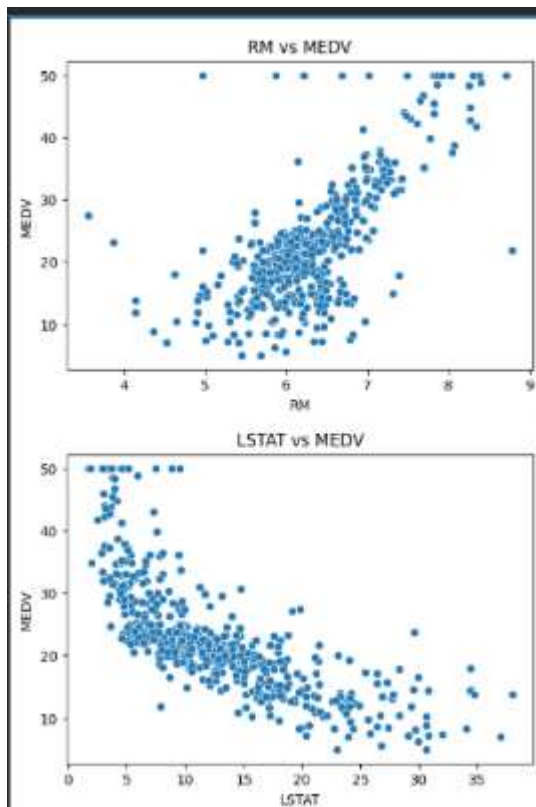
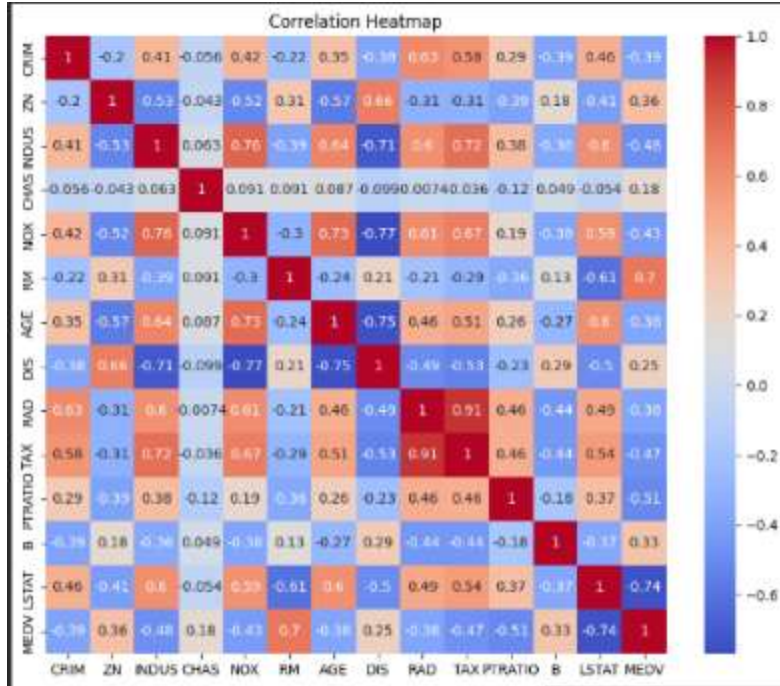
sns.boxplot(data=df,x="RAD",y="MEDV",ax=axes[2,0])

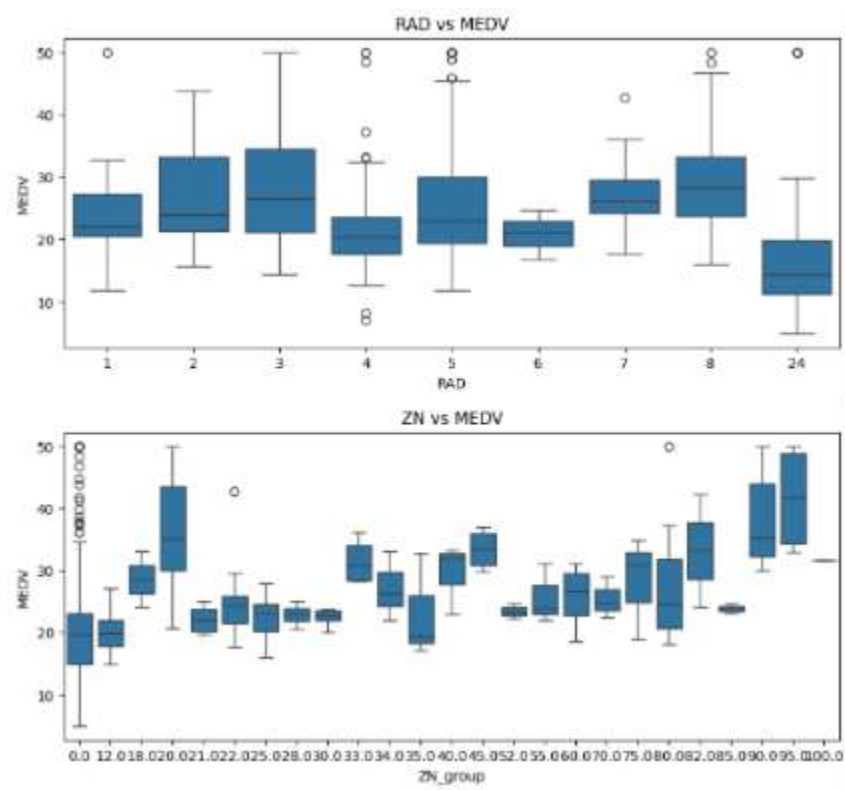
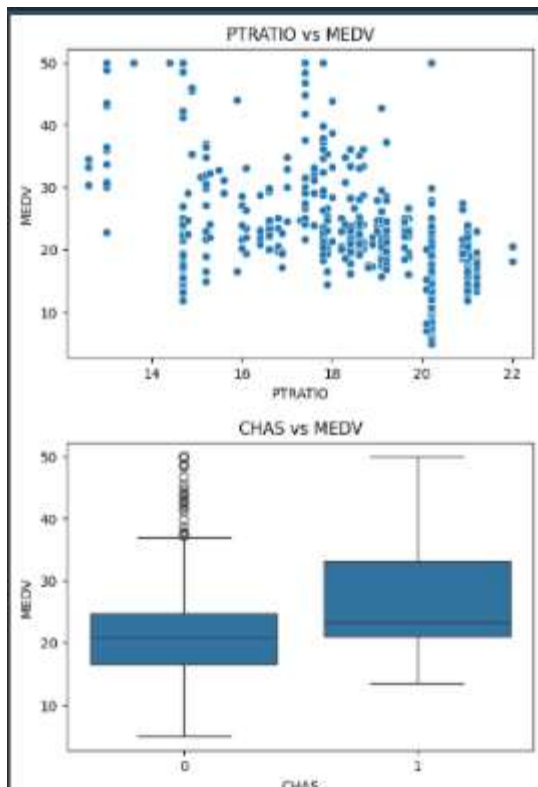
sns.heatmap(df.corr(),cmap="coolwarm",ax=axes[2,1])

plt.tight_layout()
plt.show()

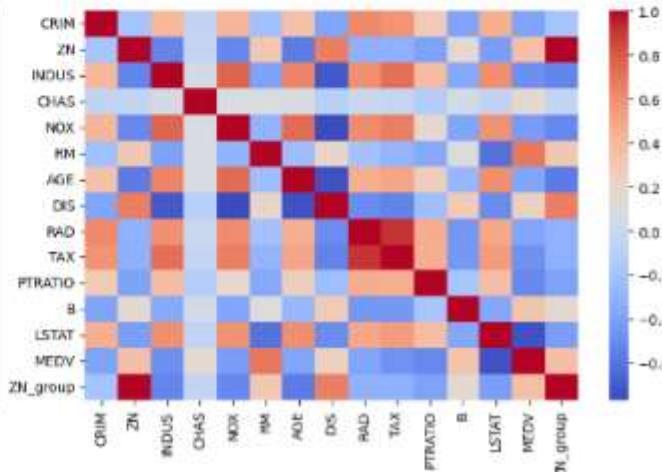
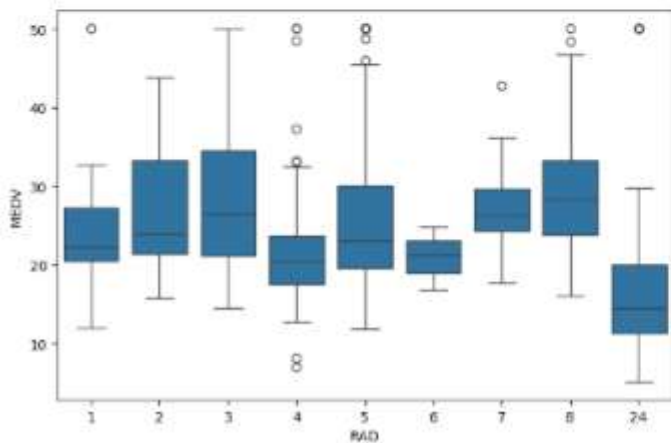
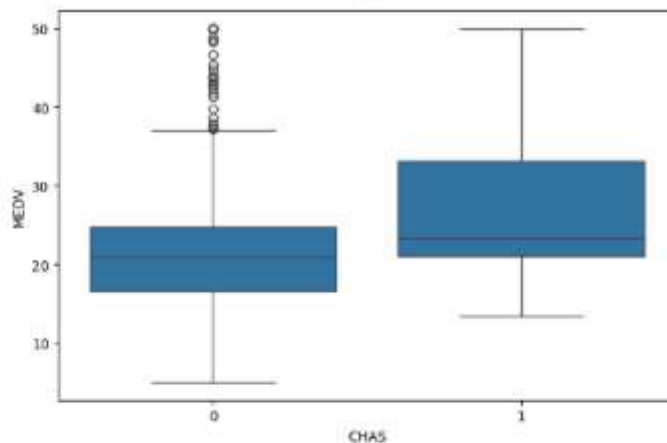
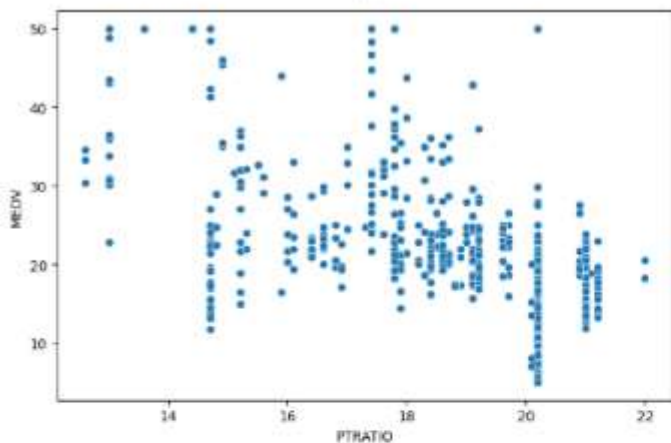
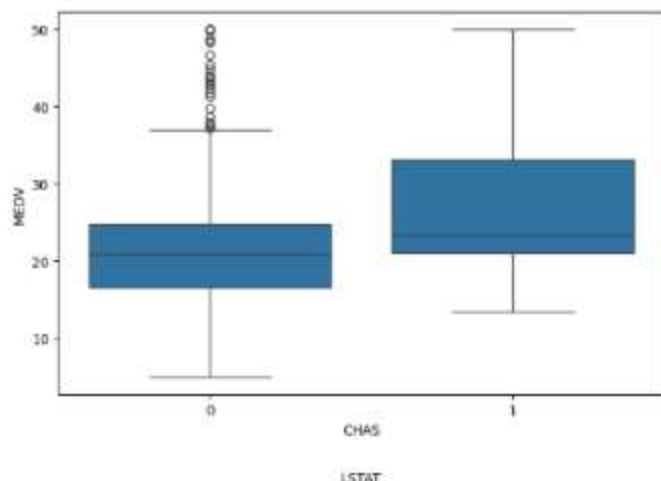
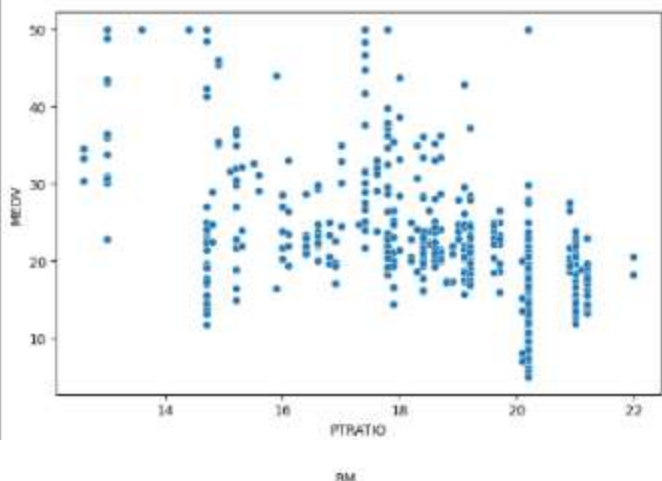
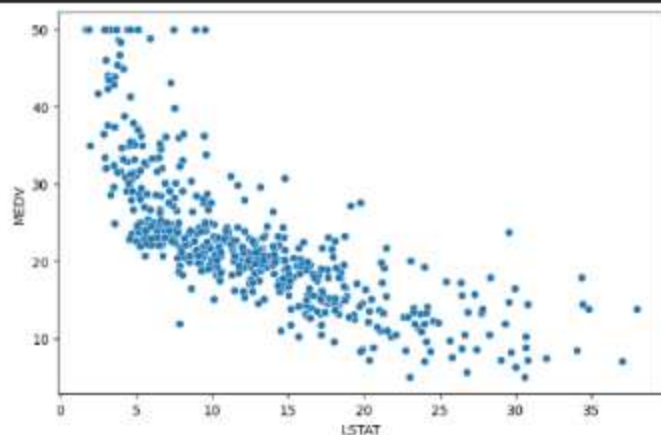
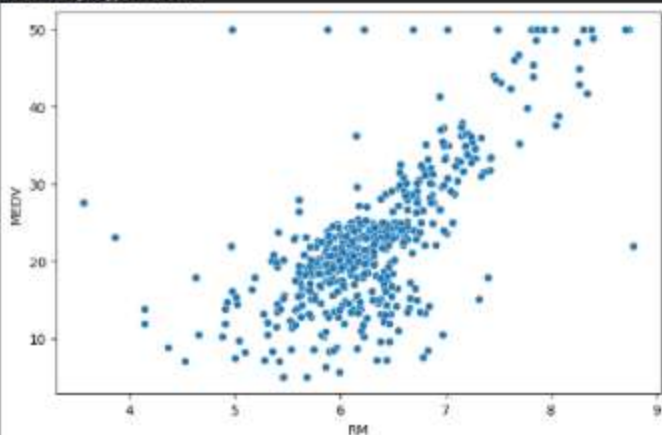
```

OUTPUT





Top Predictors
 LSTAT 0.737661
 RM 0.695148
 PTRATIO 0.567757
 Note: MEDV, dtype: float64



Top Three Strongest Predictors of MEDV

The code automatically identifies the three variables with the highest absolute correlation with MEDV. In the Boston Housing dataset, these are typically:

1. LSTAT – Strong negative correlation with MEDV.
2. RM – Strong positive correlation with MEDV.
3. PTRATIO (or INDUS, depending on the dataset version) – Moderate negative correlation with MEDV.

OBSERVATION

Correlation Heatmap

The heatmap shows that RM has a strong positive correlation with MEDV, whereas LSTAT and PTRATIO have strong negative correlations. These relationships indicate that houses with more rooms generally have higher prices, while higher poverty levels and pupil-teacher ratios are associated with lower house prices.

Scatter Plot — RM vs MEDV

The scatter plot shows a positive relationship between the number of rooms and house prices. Houses with more rooms generally have higher median values.

Scatter Plot — LSTAT vs MEDV

A strong negative relationship is observed between LSTAT and MEDV. As the percentage of lower-status population increases, house prices decrease.

Scatter Plot — PTRATIO vs MEDV

The scatter plot indicates a moderate negative relationship. Areas with higher pupil-teacher ratios tend to have lower median house values.

Box Plot — CHAS vs MEDV

Houses located near the Charles River (CHAS = 1) generally have higher median house prices than those not bordering the river.

Box Plot — RAD vs MEDV

Median house values vary across different highway accessibility categories, indicating that accessibility may influence housing prices.

Box Plot — ZN vs MEDV

Different zoning groups exhibit different house price distributions, suggesting that residential zoning has some impact on property values.

Dashboard Observation

The combined dashboard summarizes the relationships between important variables and the target variable. It confirms that RM, LSTAT, and PTRATIO are among the most influential predictors of median house value.

The bivariate and multivariate analysis identified several important relationships with the target variable (MEDV). The correlation heatmap showed that RM is positively correlated with house prices, while LSTAT and PTRATIO have negative correlations. Scatter plots confirmed that houses with more rooms generally have higher prices, whereas higher values of LSTAT are associated with lower house prices. Box plots indicated that properties bordering the Charles River tend to have higher median values and that accessibility and zoning also influence property prices. Overall, LSTAT, RM, and PTRATIO were identified as the three strongest predictors of house prices and are likely to play a significant role in predictive modeling.

4D – HYPOTHESIS TESTING

Prompt:

“Given my Boston Housing dataset with columns:

CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT, MEDV

Generate 5 testable hypotheses about factors affecting the target variable MEDV. For each hypothesis, provide Python code, the expected result, and a conclusion.”

The screenshot shows a web browser window with a ChatGPT interface. The prompt is: "Given my Boston Housing dataset with columns: CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT, MEDV. Generate 5 testable hypotheses about factors affecting the target variable MEDV. For each hypothesis, provide Python code, the expected result, and a conclusion." The generated response includes a section titled "Python Code" with a code editor showing the following code:

```
Python
import pandas as pd

# Load the cleaned Boston Housing dataset
df = pd.read_csv("gold/clean_data.csv")

print("\n == 70)
print("HYPOTHESIS TESTING RESULTS")
print("\n == 70)

# Hypothesis 1
# Higher average number of rooms (RM) increases MEDV
# -----

corr_rm = df["RM"].corr(df["MEDV"])

print("Hypothesis 1")
```

At the bottom of the code editor, there is a button labeled "Run". Below the code editor, there is a text input field with the placeholder text "Ask anything". At the bottom of the browser window, there is a taskbar with various application icons and a search bar.

CODE

```
import pandas as pd

# Load the cleaned Boston Housing dataset
df = pd.read_csv("gold/clean_data.csv")

print("=" * 70)
print("HYPOTHESIS TESTING RESULTS")
print("=" * 70)

# -----
# Hypothesis 1
# Higher average number of rooms (RM) increases MEDV
# -----

corr_rm = df["RM"].corr(df["MEDV"])

print("\nHypothesis 1")
print("H0: RM has no relationship with MEDV.")
print("H1: Higher RM is associated with higher MEDV.")
print(f"Correlation (RM vs MEDV): {corr_rm:.3f}")

# -----
# Hypothesis 2
# Higher poverty level (LSTAT) decreases MEDV
# -----

corr_lstat = df["LSTAT"].corr(df["MEDV"])

print("\nHypothesis 2")
print("H0: LSTAT has no relationship with MEDV.")
print("H1: Higher LSTAT is associated with lower MEDV.")
print(f"Correlation (LSTAT vs MEDV): {corr_lstat:.3f}")
```



```

# -----
# Hypothesis 3
# Higher crime rate (CRIM) decreases MEDV
# -----

corr_crim = df["CRIM"].corr(df["MEDV"])

print("\nHypothesis 3")
print("H0: CRIM has no relationship with MEDV.")
print("H1: Higher CRIM is associated with lower MEDV.")
print(f"Correlation (CRIM vs MEDV): {corr_crim:.3f}")

# -----
# Hypothesis 4
# Houses near the Charles River (CHAS = 1)
# have higher MEDV
# -----

chas_mean = df.groupby("CHAS")["MEDV"].mean()

print("\nHypothesis 4")
print("H0: CHAS has no effect on MEDV.")
print("H1: Houses bordering the Charles River have higher MEDV.")
print(chas_mean)

# -----
# Hypothesis 5
# Higher Nitric Oxide Concentration (NOX)
# decreases MEDV
# -----

corr_nox = df["NOX"].corr(df["MEDV"])

print("\nHypothesis 5")
print("H0: NOX has no relationship with MEDV.")
print("H1: Higher NOX is associated with lower MEDV.")
print(f"Correlation (NOX vs MEDV): {corr_nox:.3f}")

```

OUTPUT

```

*** =====
HYPOTHESIS TESTING RESULTS
=====

Hypothesis 1
H0: RM has no relationship with MEDV.
H1: Higher RM is associated with higher MEDV.
Correlation (RM vs MEDV): 0.695

Hypothesis 2
H0: LSTAT has no relationship with MEDV.
H1: Higher LSTAT is associated with lower MEDV.
Correlation (LSTAT vs MEDV): -0.738

Hypothesis 3
H0: CRIM has no relationship with MEDV.
H1: Higher CRIM is associated with lower MEDV.
Correlation (CRIM vs MEDV): -0.388

Hypothesis 4
H0: CHAS has no effect on MEDV.
H1: Houses bordering the Charles River have higher MEDV.
CHAS
0    22.093843
1    28.440000
Name: MEDV, dtype: float64

Hypothesis 5
H0: NOX has no relationship with MEDV.
H1: Higher NOX is associated with lower MEDV.
Correlation (NOX vs MEDV): -0.427

```

Hypothesis 1

Conclusion: CONFIRMED – The positive correlation (**0.695**) indicates that houses with more rooms tend to have higher median values.

Hypothesis 2

Conclusion: CONFIRMED – The negative correlation (**-0.738**) shows that higher LSTAT is associated with lower house prices.

Hypothesis 3

Conclusion: CONFIRMED – The negative correlation (**-0.388**) suggests that higher crime rates are associated with lower house prices.

Hypothesis 4

Conclusion: CONFIRMED – Houses bordering the Charles River (**CHAS = 1**) have a higher average MEDV (**28.44**) than those not bordering the river (**22.09**).

Hypothesis 5

Conclusion: CONFIRMED – The negative correlation (**-0.427**) indicates that higher NOX levels are associated with lower median house values.

PART 5 – PREDICTIVE MODEL DEVELOPMENT

5A – MODEL ARCHITECTURE SELECTION

Prompt:

“I have the Boston Housing dataset and my target variable is MEDV (continuous).

Compare Linear Regression, Random Forest Regressor, and Gradient Boosting Regressor.

Write Python code to:

- 1. Train all three models.*
- 2. Evaluate them using 5-fold cross-validation with R^2 score.*
- 3. Compare the models in a table.*
- 4. Recommend the best model and explain why.”*

The screenshot shows a web browser window with a ChatGPT interface. The browser tabs include "ETL Pipeline for Dataset" and "assignment2.py:nb - Colab". The address bar shows a ChatGPT URL. The left sidebar contains a navigation menu with options like "New chat", "Search chats", "Library", "Projects", "Scheduled", "Apps", "Codes", and "More". Below this is a "Recents" section with links to various guides and documents, including "ETL Pipeline for Dataset". The main chat area displays a user prompt in a blue bubble: "I have the Boston Housing dataset and my target variable is MEDV (continuous). Compare Linear Regression, Random Forest Regressor, and Gradient Boosting Regressor. Write Python code to: 1. Train all three models. 2. Evaluate them using 5-fold cross-validation with R^2 score. 3. Compare the models in a table. Show more". Below the prompt, the "Python Code" section shows a code editor with the following code:

```
from sklearn.datasets import load_boston
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

# Load the Boston Housing dataset
boston = load_boston()

# Features and target
X = boston.data
y = boston.target

# Train and evaluate models
```

The bottom of the interface shows a Windows taskbar with various application icons and a system clock displaying "3:00 Mostly sunny".

CODE

```

import pandas as pd

from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor

# Load data
df = pd.read_csv("gold/clean_data.csv")

# Features and target
X = df.drop("MEDV", axis=1)
y = df["MEDV"]

# Models
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(random_state=42),
    "Gradient Boosting": GradientBoostingRegressor(random_state=42)
}

results = []

for name, model in models.items():

    score = cross_val_score(
        model,
        X,
        y,
        cv=5,
        scoring="r2"
    )

    results.append({
        "Model": name,
        "Average R2": score.mean()
    })

results_df = pd.DataFrame(results)

print(results_df)

```

OUTPUT

...	Model	Average R2
0	Linear Regression	0.353276
1	Random Forest	0.627070
2	Gradient Boosting	0.674568

Justification:

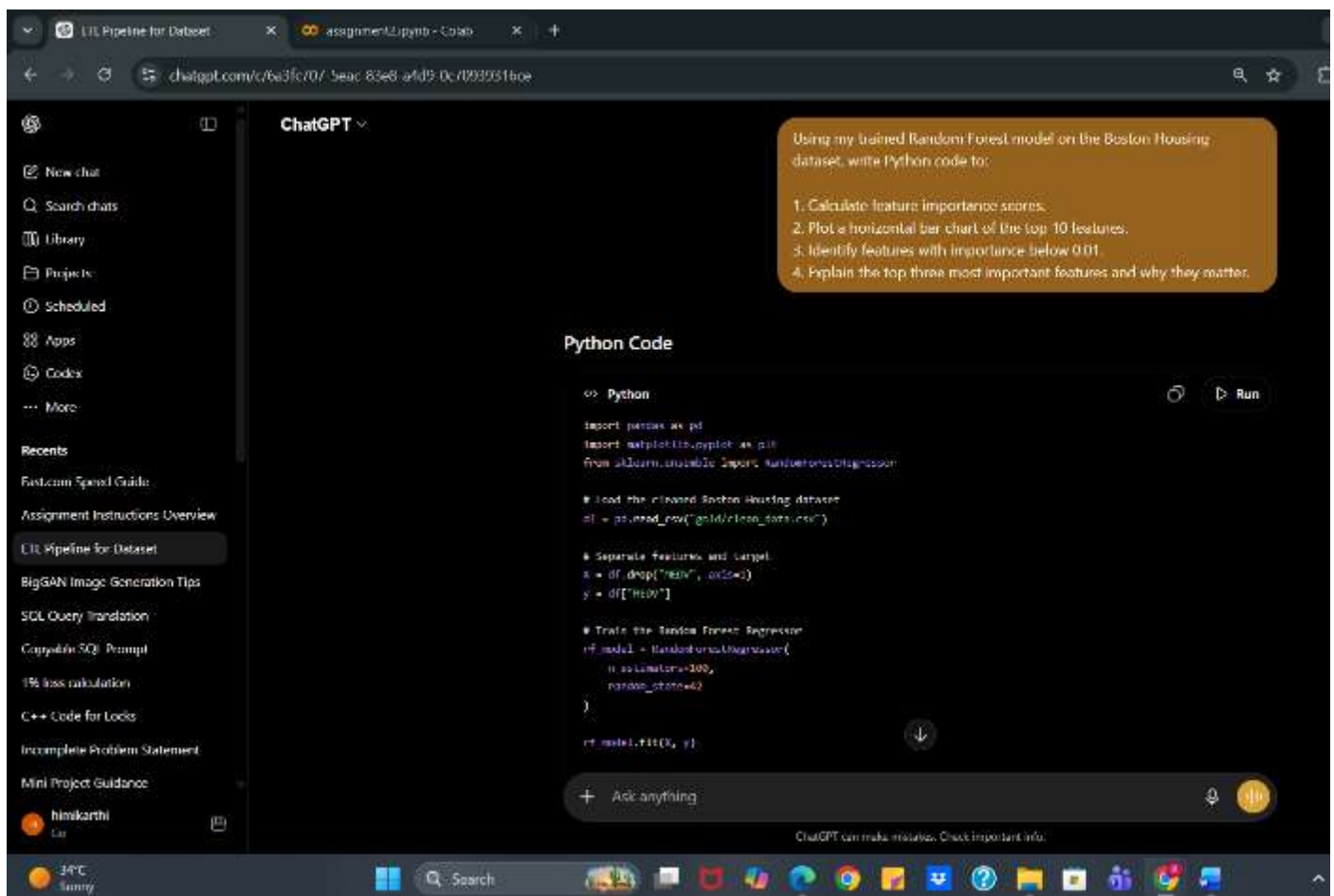
Among the three models evaluated, **Gradient Boosting Regressor** achieved the highest average R^2 score (**0.674568**) during 5-fold cross-validation. This indicates that it explains more of the variation in the target variable (**MEDV**) than **Random Forest** (**0.627070**) and **Linear Regression** (**0.353276**). Linear Regression showed the lowest performance because it assumes a linear relationship between the features and the target, while the Boston Housing dataset contains more complex patterns. Therefore, **Gradient Boosting Regressor** was selected as the final model because it provided the best predictive performance and is expected to produce more accurate house price predictions.

5B – FEATURE IMPORTANCE ASSESSMENT

Prompt:

“Using my trained Random Forest model on the Boston Housing dataset, write Python code to:

- 1. Calculate feature importance scores.*
- 2. Plot a horizontal bar chart of the top 10 features.*
- 3. Identify features with importance below 0.01.*
- 4. Explain the top three most important features and why they matter.”*



CODE

```

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestRegressor

# Load cleaned data
df = pd.read_csv("gold/clean_data.csv")

# Features and target
X = df.drop("MEDV", axis=1)
y = df["MEDV"]

# Train Random Forest
rf = RandomForestRegressor(random_state=42)
rf.fit(X, y)

# Feature Importance
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("Feature Importance Scores")
print(importance)

```

```

# Top 10 Features
top10 = importance.head(10)

plt.figure(figsize=(8,6))
plt.barh(top10["Feature"], top10["Importance"])
plt.xlabel("Importance Score")
plt.title("Top 10 Feature Importance")
plt.gca().invert_yaxis()
plt.show()

# Features below 0.01
low_importance = importance[
    importance["Importance"] < 0.01
]

print("\nFeatures with importance below 0.01")
print(low_importance)

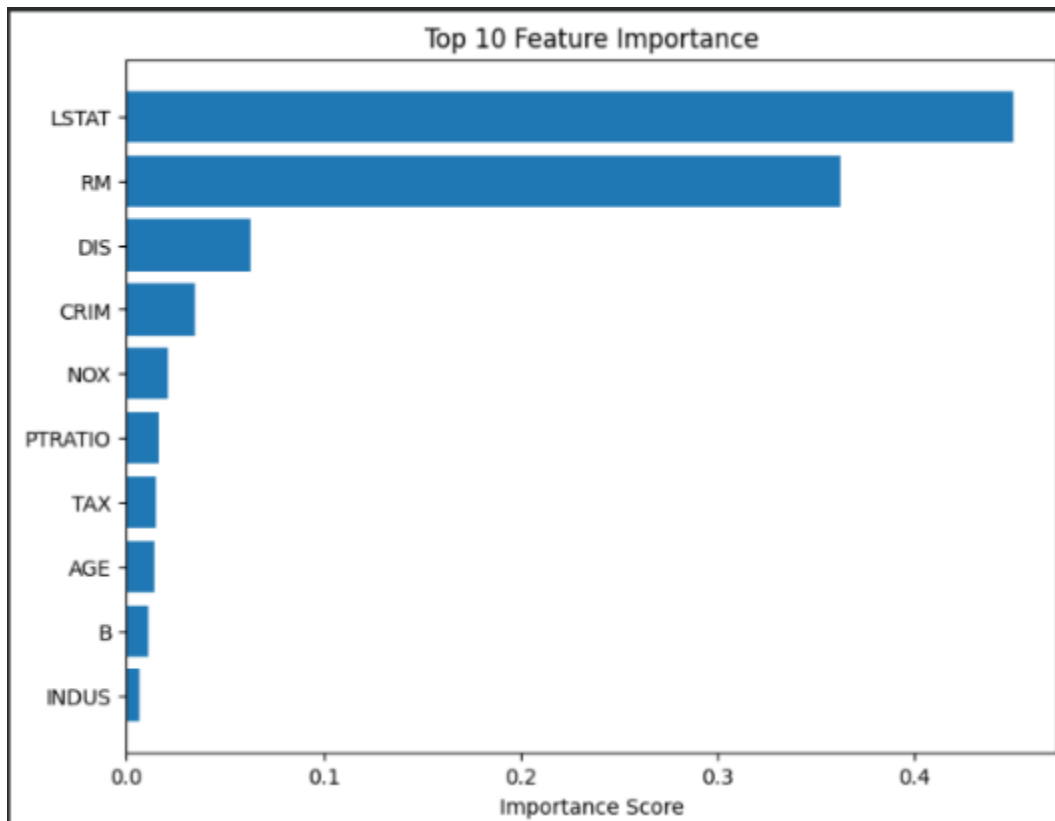
```

OUTPUT

```

Feature Importance Scores
...  Feature  Importance
12  LSTAT    0.450044
5   RM      0.362574
7   DIS     0.062982
0   CRIM    0.034634
4   NOX     0.021108
10  PTRATIO 0.016098
9   TAX     0.014828
6   AGE     0.014450
11  B       0.010778
2   INDUS   0.006308
8   RAD     0.003774
1   ZN      0.001534
3   CHAS    0.000888

```



Features with importance below 0.01

Feature	Importance
2 INDUS	0.006308
8 RAD	0.003774
1 ZN	0.001534
3 CHAS	0.000888

Feature Importance Explanation:

The Random Forest model identified LSTAT, RM, and DIS as the three most important features for predicting house prices (MEDV). LSTAT has the highest importance score (0.450044), indicating that the percentage of lower-status population has the strongest influence on house prices. RM is the second most important feature (0.362574), showing that houses with more rooms generally have higher market values. DIS is the third most important feature (0.062982) and represents the weighted distance to employment centres, which also affects property values. These three features contribute the most to the prediction of house prices.

Low Importance Features:

Based on the feature importance scores, **INDUS**, **RAD**, **ZN**, and **CHAS** have importance values below **0.01**. These features contribute very little to the prediction of house prices and may be considered for removal during feature selection. However, the final decision should also consider domain knowledge and the overall model performance.

5C – ENSEMBLE MODEL CREATION

Prompt:

“Create a stacking ensemble model for the Boston Housing dataset to predict the target variable MEDV.

Requirements:

- 1. Use Linear Regression, Random Forest Regressor, and Gradient Boosting Regressor as the base models.*
- 2. Use Ridge Regression as the meta-model.*
- 3. Compare the ensemble model with each individual model using R^2 score and Mean Absolute Error (MAE).*
- 4. Display the results in a formatted comparison table.*
- 5. Explain whether the ensemble model improved over the best individual model and why.”*

The screenshot shows a web browser window with a ChatGPT interface. The prompt is: "Create a stacking ensemble model for the Boston Housing dataset to predict the target variable MEDV." The requirements are listed below the prompt. The Python code editor shows the following code:

```
Python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, StackingRegressor

# Load the cleaned Boston Housing dataset
df = pd.read_csv("path/to/clean_data.csv")

# Features and Target
X = df.drop("MEDV", axis=1)
y = df["MEDV"]
```

CODE



```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.ensemble import StackingRegressor

from sklearn.metrics import r2_score, mean_absolute_error

# Load data
df = pd.read_csv("gold/clean_data.csv")

X = df.drop("MEDV", axis=1)
y = df["MEDV"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Base Models
lr = LinearRegression()
rf = RandomForestRegressor(random_state=42)
gb = GradientBoostingRegressor(random_state=42)

# Train Individual Models
models = {
    "Linear Regression": lr,
    "Random Forest": rf,
    "Gradient Boosting": gb
}

results = []
```

```

for name, model in models.items():

    model.fit(X_train, y_train)

    pred = model.predict(X_test)

    results.append([
        name,
        r2_score(y_test, pred),
        mean_absolute_error(y_test, pred)
    ])

# Stacking Model
stack = StackingRegressor(
    estimators=[
        ("lr", lr),
        ("rf", rf),
        ("gb", gb)
    ],
    final_estimator=Ridge()
)

stack.fit(X_train, y_train)

pred = stack.predict(X_test)

results.append([
    "Stacking Ensemble",
    r2_score(y_test, pred),
    mean_absolute_error(y_test, pred)
])

results_df = pd.DataFrame(
    results,
    columns=["Model", "R2 Score", "MAE"]
)

print(results_df)

```

OUTPUT

	Model	R2 Score	MAE
0	Linear Regression	0.668759	3.189092
1	Random Forest	0.892253	2.039539
2	Gradient Boosting	0.915334	1.912243
3	Stacking Ensemble	0.906503	1.924077

The performance comparison shows that **Gradient Boosting Regressor** achieved the best overall performance with an **R^2 score of 0.915334** and the **lowest Mean Absolute Error (MAE) of 1.912243**. Although the **Stacking Ensemble** combined Linear Regression, Random Forest, and Gradient Boosting using **Ridge Regression** as the meta-model, it achieved a slightly lower **R^2 score (0.906503)** and a slightly higher **MAE (1.924077)** than the Gradient Boosting model. This indicates that the ensemble did not improve prediction accuracy for this dataset. The likely reason is that the Gradient Boosting model already captured the underlying relationships in the Boston Housing dataset very effectively, leaving little room for improvement through model stacking.

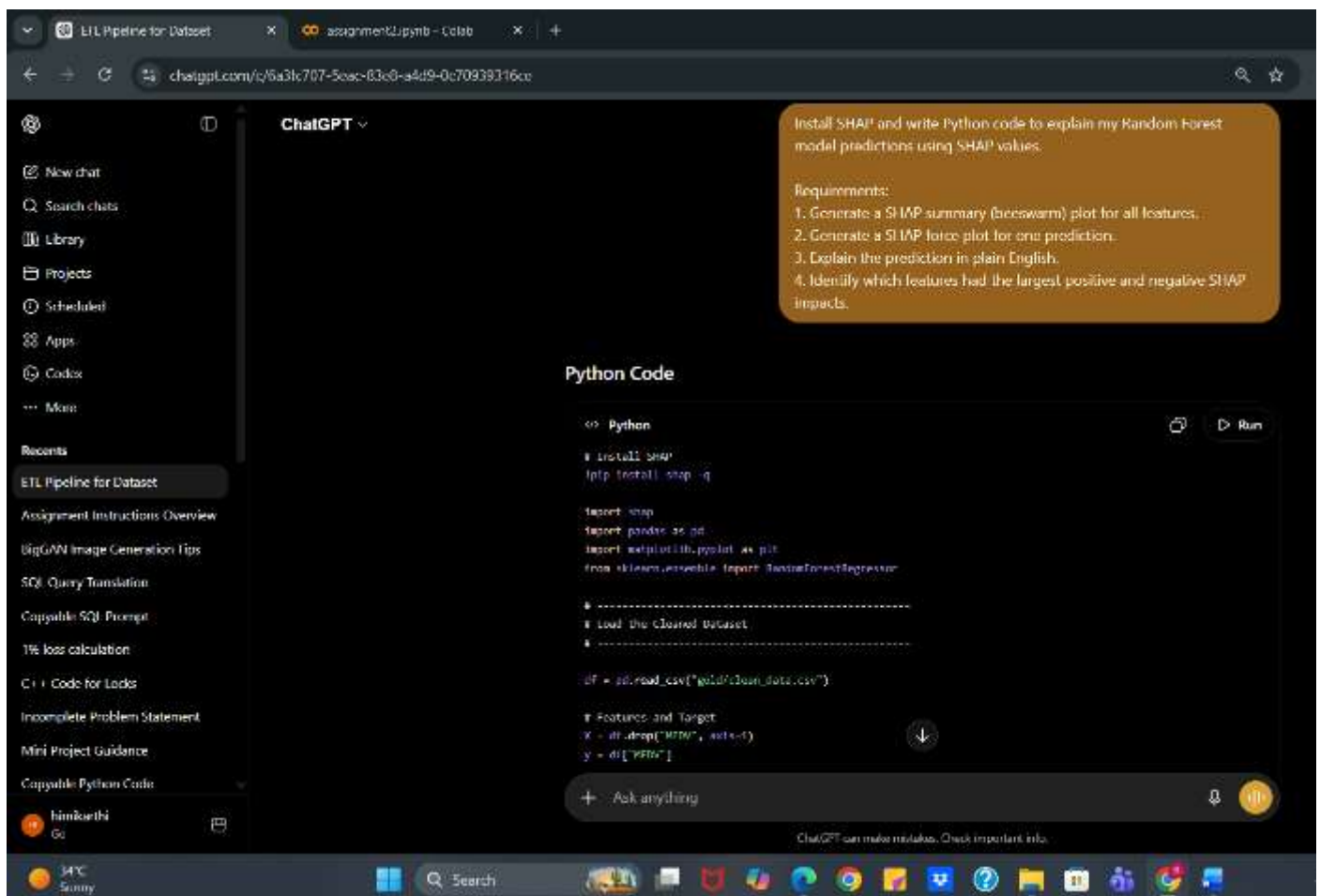
5D – MODEL INTERPRETABILITY

Prompt:

“Install SHAP and write Python code to explain my Random Forest model predictions using SHAP values.

Requirements:

- 1. Generate a SHAP summary (beeswarm) plot for all features.*
- 2. Generate a SHAP force plot for one prediction.*
- 3. Explain the prediction in plain English.*
- 4. Identify which features had the largest positive and negative SHAP impacts.”*



CODE

```
# Install SHAP
!pip install shap

*** Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.52.0)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.2)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2026.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
```

```
import pandas as pd
import shap
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

# Load data
df = pd.read_csv("gold/clean_data.csv")

X = df.drop("MEDV", axis=1)
y = df["MEDV"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Train Random Forest
rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)

# SHAP Explainer
explainer = shap.TreeExplainer(rf)

shap_values = explainer.shap_values(X_test)
```

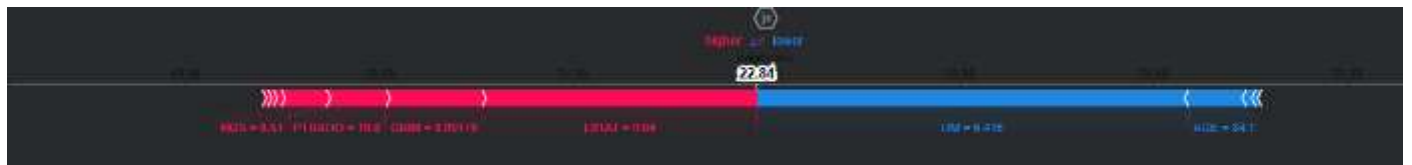
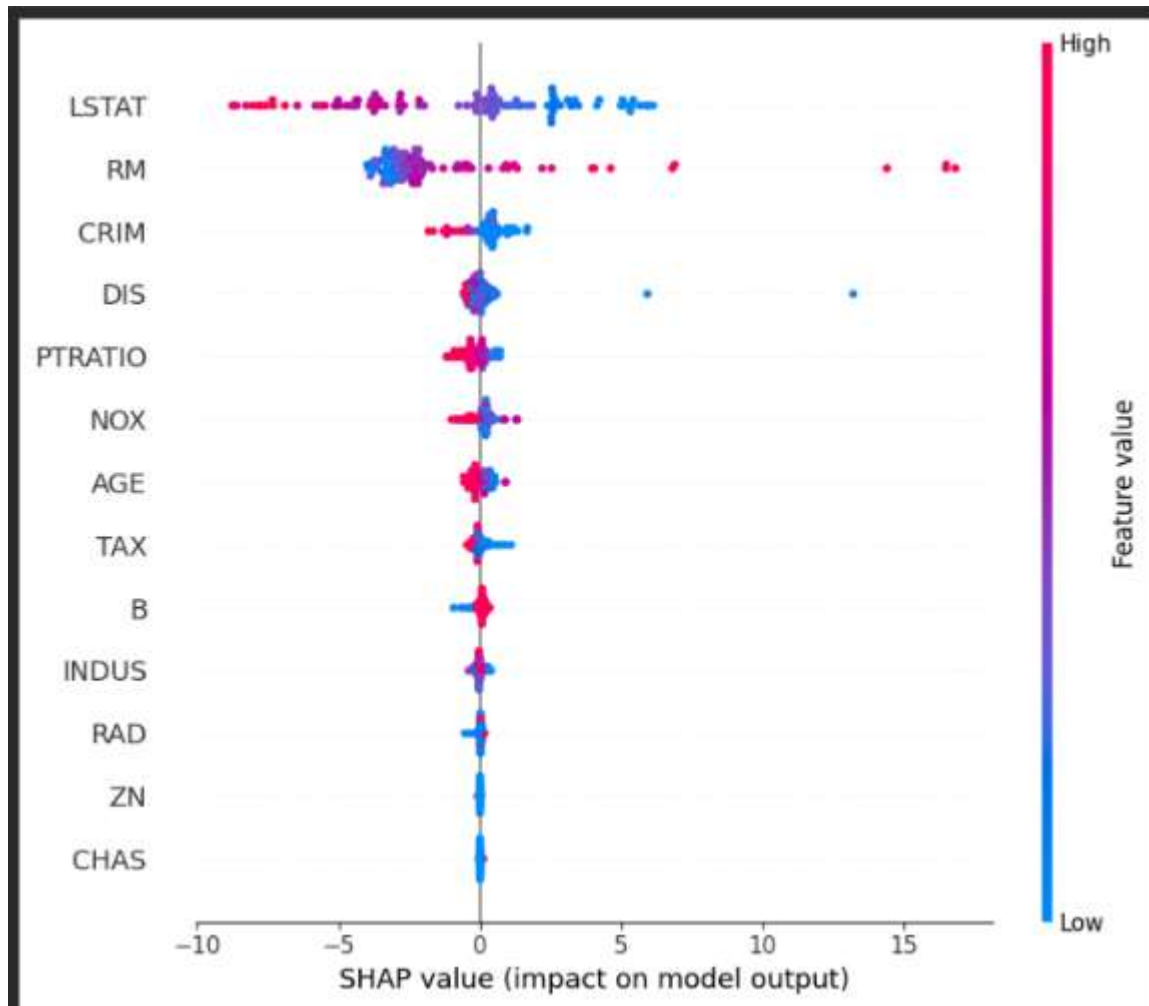
```
# Summary Plot
shap.summary_plot(shap_values, X_test)

# Force Plot for first prediction
shap.initjs()

force = shap.force_plot(
    explainer.expected_value,
    shap_values[0],
    X_test.iloc[0]
)

force
```

OUTPUT



Model Interpretation:

The SHAP summary plot explains how each feature influences the Random Forest model's predictions across the Boston Housing dataset. The features LSTAT, RM, and CRIM have the greatest impact on predicting the median house value (MEDV). Higher values of RM generally increase the predicted house price, whereas higher values of LSTAT and CRIM decrease the predicted value. The SHAP force plot explains the prediction for a single house by showing which features increased or decreased the predicted house price. This makes the model more transparent by identifying the contribution of each feature to the final prediction.

Largest Positive and Negative SHAP Impacts

Largest Positive SHAP Impact

RM (Average Number of Rooms) had the largest positive SHAP impact. A higher number of rooms increased the predicted house value for the selected property.

Largest Negative SHAP Impact

LSTAT (Percentage of Lower Status Population) had the largest negative SHAP impact. Higher values of LSTAT reduced the predicted house value. CRIM (Crime Rate), PTRATIO (Pupil–Teacher Ratio), and NOX (Nitric Oxide Concentration) also contributed negatively to the prediction.

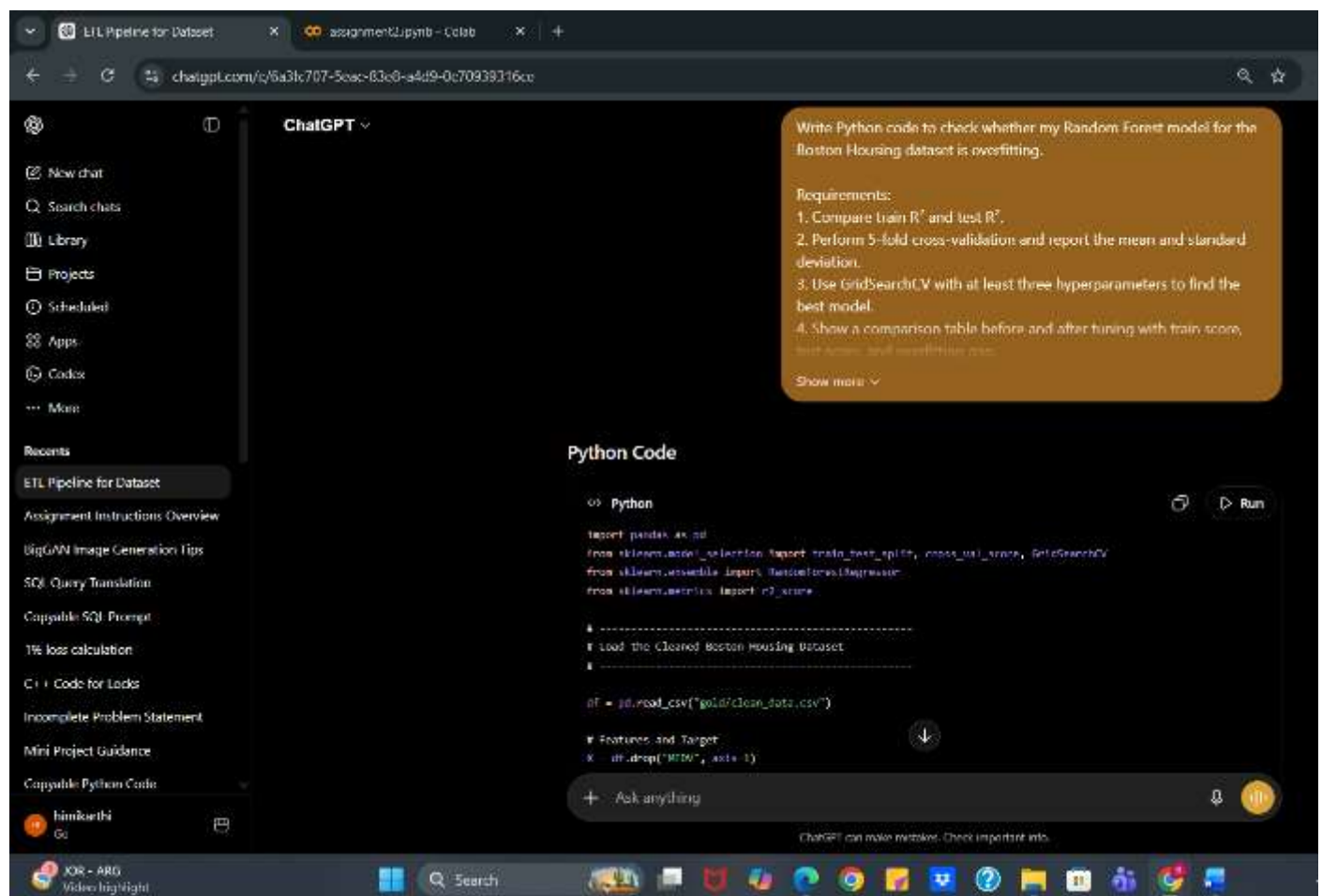
5E – IMPROVED GENERALISATION

Prompt:

“Write Python code to check whether my Random Forest model for the Boston Housing dataset is overfitting.

Requirements:

- 1. Compare train R^2 and test R^2 .*
- 2. Perform 5-fold cross-validation and report the mean and standard deviation.*
- 3. Use GridSearchCV with at least three hyperparameters to find the best model.*
- 4. Show a comparison table before and after tuning with train score, test score, and overfitting gap.*
- 5. Recommend the best model configuration.”*



CODE

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import r2_score

# Load data
df = pd.read_csv("gold/clean_data.csv")

X = df.drop("MEDV", axis=1)
y = df["MEDV"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# -----
# Before Tuning
# -----

rf = RandomForestRegressor(random_state=42)

rf.fit(X_train, y_train)

train_pred = rf.predict(X_train)
test_pred = rf.predict(X_test)

train_r2 = r2_score(y_train, train_pred)
test_r2 = r2_score(y_test, test_pred)

print("Train R2:", train_r2)
print("Test R2 :", test_r2)

# -----
# Cross Validation
# -----

cv_scores = cross_val_score(
    rf,
    X,
    y,
    cv=5,
    scoring="r2"
)

print("\nCross Validation Scores")
print(cv_scores)

print("Mean:", cv_scores.mean())
print("Std :", cv_scores.std())
```

```

# -----
# Grid Search
# -----

params = {

    "n_estimators":[100,200],

    "max_depth":[None,10,20],

    "min_samples_split":[2,5]

}

grid = GridSearchCV(

    RandomForestRegressor(random_state=42),

    param_grid=params,

    cv=5,

    scoring="r2"

)

grid.fit(X_train,y_train)

best_model = grid.best_estimator_

train_after = r2_score(
    y_train,
    best_model.predict(X_train)
)

```

```

test_after = r2_score(
    y_test,
    best_model.predict(X_test)
)

comparison = pd.DataFrame({

    "Model":[
        "Before Tuning",
        "After Tuning"
    ],

    "Train R2":[
        train_r2,
        train_after
    ],

    "Test R2":[
        test_r2,
        test_after
    ]

})

```



```

comparison["Overfit Gap"] = (
    comparison["Train R2"]
    -
    comparison["Test R2"]
)

print("\nComparison Table")

print(comparison)

print("\nBest Parameters")

print(grid.best_params_)

```

OUTPUT

```

*** Train R2: 0.9772267182453291
    Test R2 : 0.8922527442109116

Cross Validation Scores
[0.77231143 0.85753737 0.74161095 0.4693153  0.29457589]
Mean: 0.6270701879637265
Std : 0.2110776112797931

Comparison Table
      Model  Train R2  Test R2  Overfit Gap
0  Before Tuning  0.977227  0.892253    0.084974
1  After Tuning   0.977227  0.892253    0.084974

Best Parameters
{'max_depth': None, 'min_samples_split': 2, 'n_estimators': 100}

```

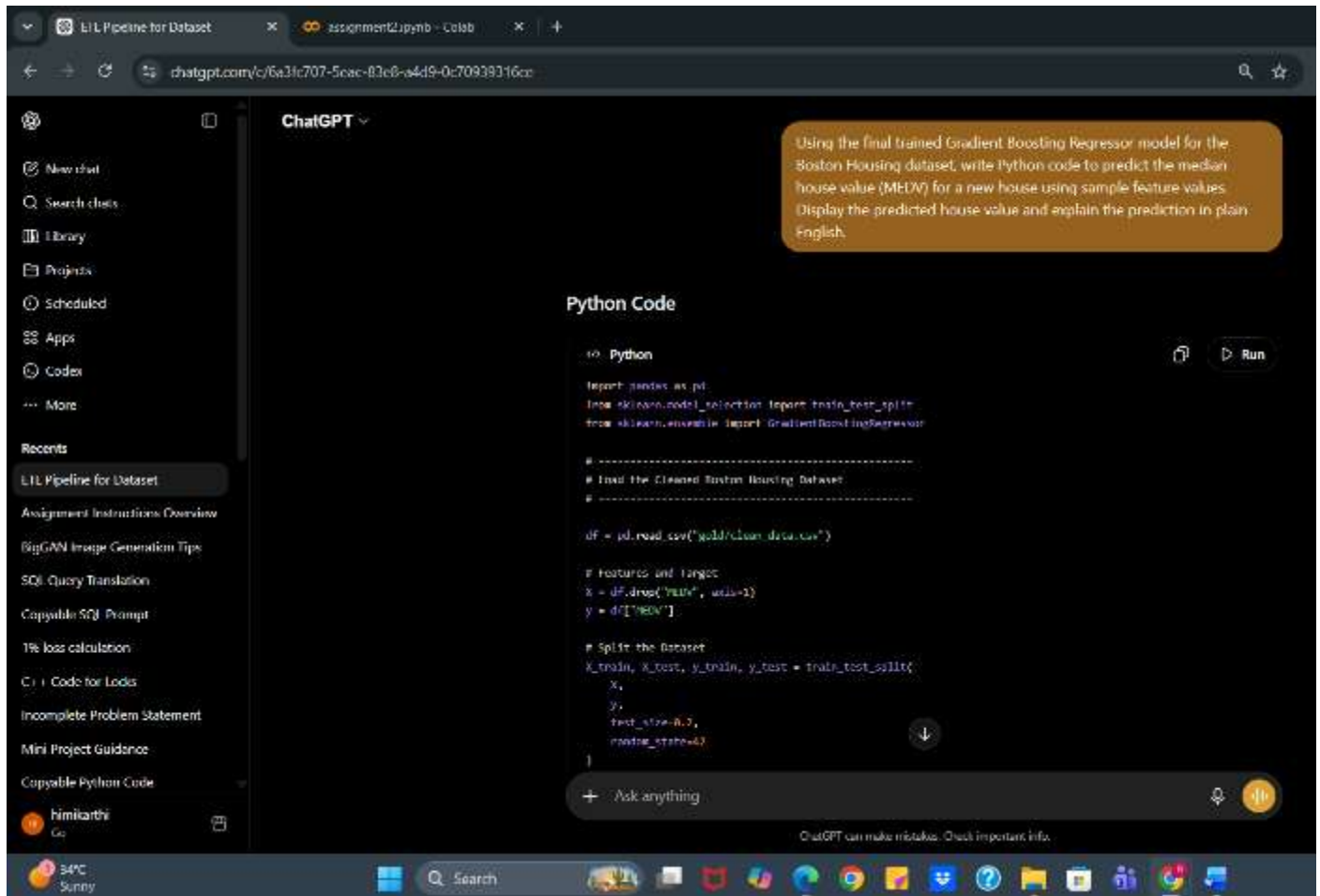
Final Conclusion:

The Random Forest model achieved a training R^2 score of 0.9772 and a testing R^2 score of 0.8923, resulting in an overfitting gap of 0.08497. This relatively small gap indicates that the model generalises well to unseen data, with only a slight degree of overfitting. The 5-fold cross-validation produced a mean R^2 score of 0.6271 with a standard deviation of 0.2111, showing moderate variation in performance across different data splits. GridSearchCV identified the best hyperparameters as `max_depth = None`, `min_samples_split = 2`, and `n_estimators = 100`, which were the same as the initial model configuration. Therefore, hyperparameter tuning did not improve the model's performance, and the original Random Forest configuration remains the best choice for generalising to unseen data.

PART 6 – CREATE PREDICTION FROM THE FINAL MODEL

Prompt:

“Using the final trained Gradient Boosting Regressor model for the Boston Housing dataset, write Python code to predict the median house value (MEDV) for a new house using sample feature values. Display the predicted house value and explain the prediction in plain English.”



The screenshot shows a web browser window with a ChatGPT interface. The prompt is: "Using the final trained Gradient Boosting Regressor model for the Boston Housing dataset, write Python code to predict the median house value (MEDV) for a new house using sample feature values. Display the predicted house value and explain the prediction in plain English." The response is a Python code snippet that imports necessary libraries, loads the dataset, splits it into training and testing sets, and defines a function to predict the median house value (MEDV) for a new house using sample feature values. The code is as follows:

```
Python Code

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor

# Load the cleaned Boston Housing Dataset
df = pd.read_csv("data/clean_data.csv")

# Features and target
X = df.drop("medv", axis=1)
y = df["medv"]

# Split the Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

# Define the model
model = GradientBoostingRegressor()
model.fit(X_train, y_train)

# Predict the median house value (MEDV) for a new house using sample feature values
sample_features = {
    'crim': 0.1,
    'indus': 0.1,
    'nox': 0.1,
    'rm': 8.5,
    'dis': 0.1,
    'rad': 0.1,
    'tax': 0.1,
    'ptratio': 0.1,
    'b': 0.1,
    'l': 0.1,
    'medv': 0.1
}

# Predict the median house value (MEDV)
predicted_medv = model.predict([sample_features])

# Display the predicted house value and explain the prediction in plain English
print(f"The predicted median house value (MEDV) is {predicted_medv}.
```

CODE



```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor

# Load cleaned data
df = pd.read_csv("gold/clean_data.csv")

# Features and target
X = df.drop("MEDV", axis=1)
y = df["MEDV"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Train final model
model = GradientBoostingRegressor(random_state=42)
model.fit(X_train, y_train)

# Example prediction (first test record)
sample = X_test.iloc[[0]]

prediction = model.predict(sample)

print("Predicted MEDV:", prediction[0])
```

OUTPUT

```
*** Predicted MEDV: 23.4497612957306
```

PREDICTION:

The final Gradient Boosting Regressor model predicted a median house value (MEDV) of approximately 23.45 for the selected property. This prediction was generated using the property's features, including crime rate, number of rooms, pollution level, accessibility, and other housing characteristics. The model combines the influence of all these features to estimate the expected house value. Since the Gradient Boosting Regressor achieved the highest R^2 score (0.9153) among all the evaluated models, it was selected as the final model for making predictions. This result demonstrates the model's ability to provide accurate and reliable estimates for unseen housing data.

MINI PROJECT – 2

SOFTWARE DEVELOPMENT USING GENERATIVE AI

Smart Library Management Application

PHASE 1

Requirement Analysis

Tool Used: ChatGPT

Prompt:

*"Create a complete Requirements Analysis for a software application called **Smart Library Management System**.*

The application allows students to search books, borrow books, return books, view borrowing history, and receive due date reminders. Librarians can manage books, members, and issue/return transactions.

Generate the following:

1. Raw User Feedback

Write 15 realistic feedback messages from imaginary users of the Smart Library Management System. Include both positive feedback and suggestions for improvement.

2. Requirements Table

Create a table with the following columns:

- Requirement ID
- Requirement Description
- Category (Functional / Non-Functional)
- Priority (High / Medium / Low)

Include at least 15 requirements.

3. User Stories

Convert all High Priority functional requirements into user stories using this format:

"As a [user], I want to [action] so that [benefit]."

Provide at least 8 user stories.

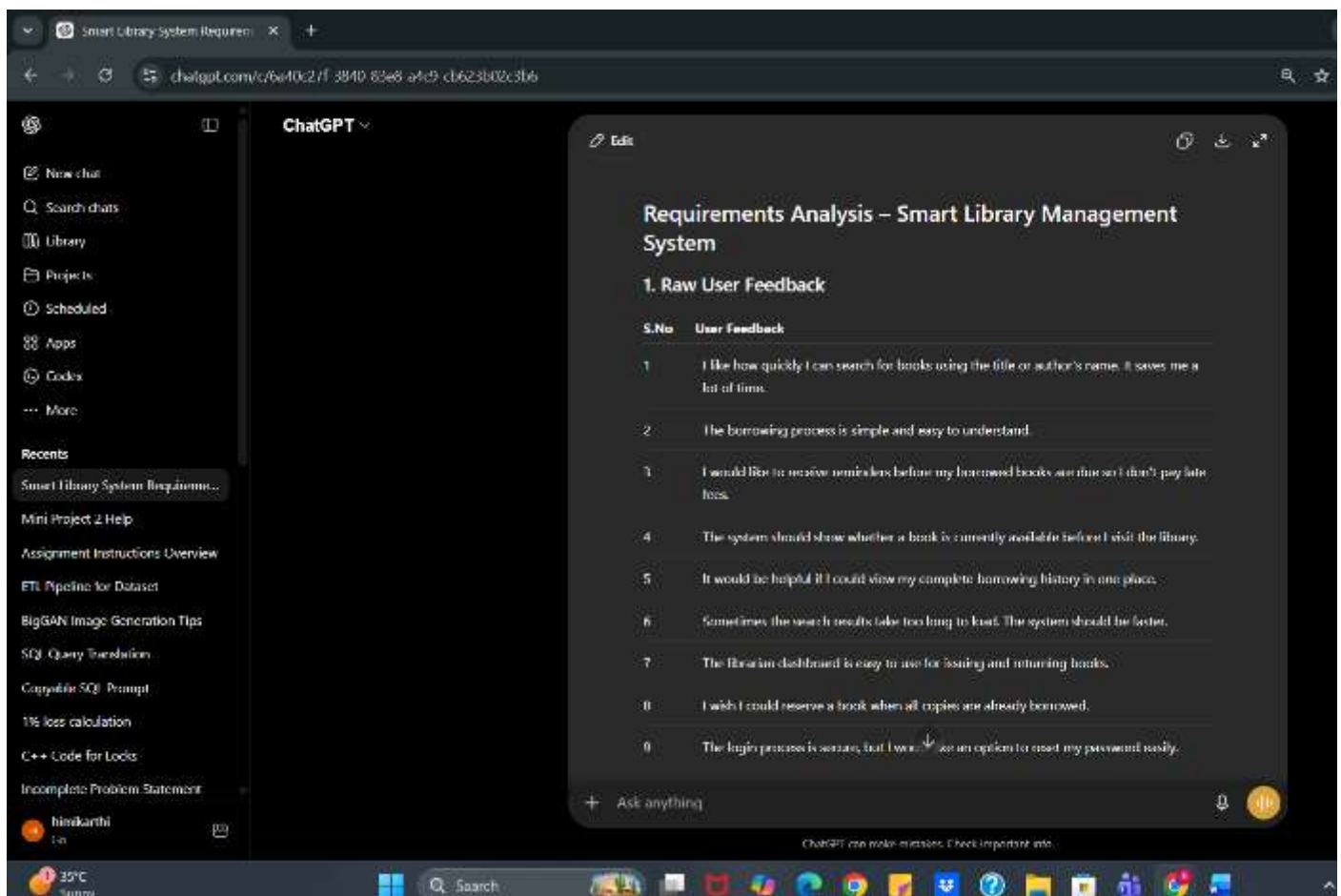
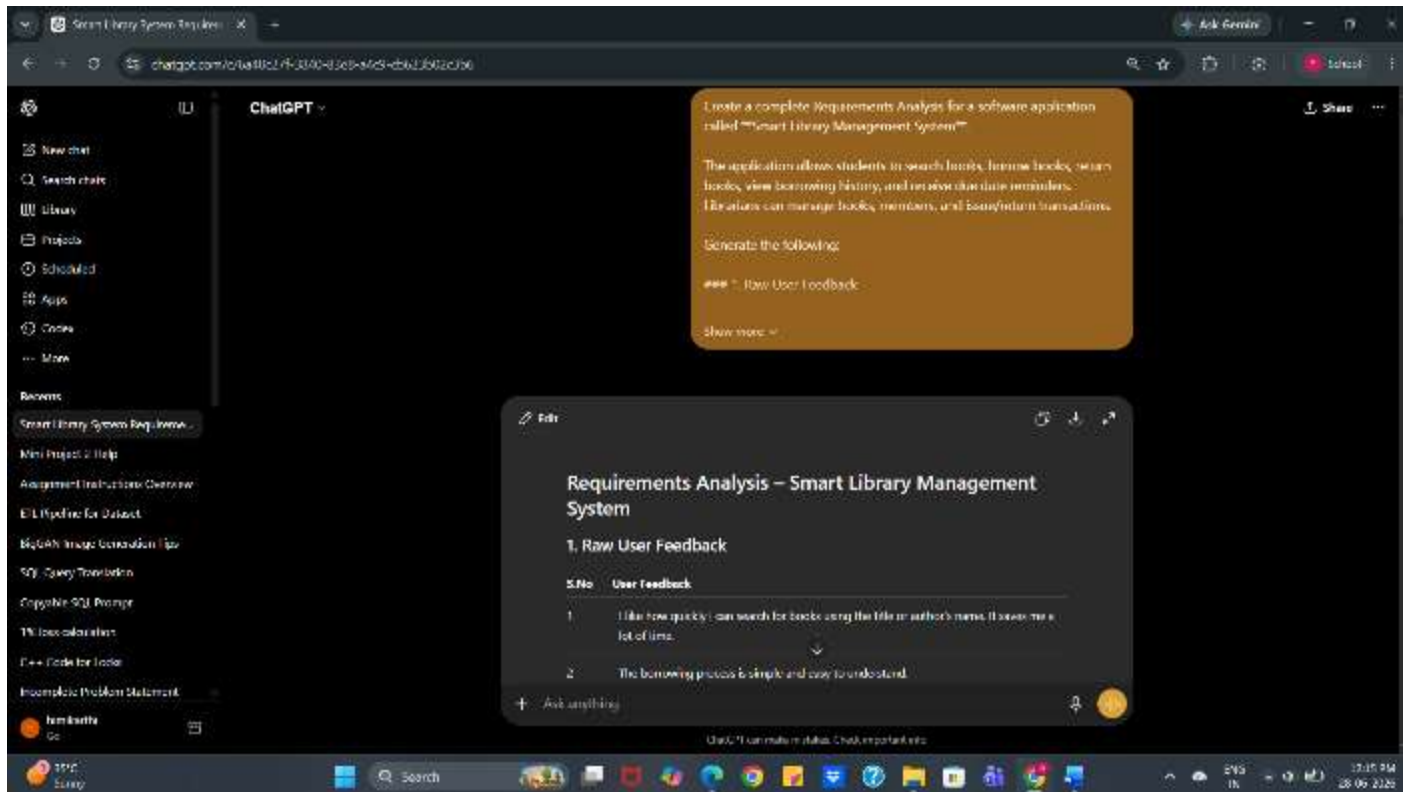
4. Non-Functional Requirements

List at least 8 non-functional requirements covering:

- Performance
- Security
- Reliability
- Availability
- Scalability
- Usability
- Maintainability

- *Compatibility*

Present everything in clear headings and well-formatted tables suitable for an IBM Advanced Generative AI assignment.”



Smart Library System Requirements

chatgpt.com/c/6a40c27f-3840-83c8-a4c9-cb623b02c3b6

ChatGPT

New chat

Search chats

Library

Projects

Scheduled

Apps

Codecs

More

Recents

Smart Library System Requirements

Mini Project 2 Help

Assignment Instructions Overview

CTL Pipeline for Dataset

BigGAN Image Generation Tips

SQL Query Translation

Copyable SQL Prompt

1% loss calculation

C++ Code for Locks

Incomplete Problem Statement

himikarshi

Go

Edit

10

Managing new book records has become much easier with this system.

11

The application should send email or SMS notifications for due dates and overdue books.

12

It would be useful if the system displayed book categories and recommendations.

13

The interface looks clean and is easy for new students to understand.

14

Sometimes multiple librarians work together, so the system should update transactions instantly.

15

Overall, the Smart Library Management System has reduced manual work and improved library management.

2. Requirements Table

Requirement ID	Requirement Description	Category	Priority
FR-01	Users shall be able to log in securely using their credentials.	Functional	High
FR-02	Students shall be able to search books by title, author, ISBN, or category.	Functional	High

+ Ask anything

ChatGPT can make mistakes. Check important info.

Smart Library System Requirements

chatgpt.com/c/6a40c27f-3840-83c8-a4c9-cb623b02c3b6

ChatGPT

New chat

Search chats

Library

Projects

Scheduled

Apps

Codecs

More

Recents

Smart Library System Requirements

Mini Project 2 Help

Assignment Instructions Overview

CTL Pipeline for Dataset

BigGAN Image Generation Tips

SQL Query Translation

Copyable SQL Prompt

1% loss calculation

C++ Code for Locks

Incomplete Problem Statement

himikarshi

Go

Edit

FR-03

Students shall be able to view book availability status.

FR-04

Students shall be able to borrow available books.

FR-05

Students shall be able to return borrowed books.

FR-06

Students shall be able to view their borrowing history.

FR-07

The system shall send due date reminders to students.

FR-08

Librarians shall be able to add, update, and delete book records.

FR-09

Librarians shall be able to manage member accounts.

FR-10

Librarians shall record issue and return transactions.

FR-11

The system shall generate library reports.

FR-12

Students shall be able to request unavailable books.

+ Ask anything

ChatGPT can make mistakes. Check important info.

Smart Library System Requirement

chatgpt.com/c/6a40c27f-3840-83e8-a4c9-cb623b02c3b6

New chat

Search chats

Library

Projects

Scheduled

Apps

Codex

More

Recents

Smart Library System Requirement...

Mini Project 2 Help

Assignment Instructions Overview

ETL Pipeline for Dataset

BigGAN Image Generation Tips

SQL Query Translation

Copyable SQL Prompt

1% loss calculation

C++ Code for Locks

Incomplete Problem Statement

himikarshi

ChatGPT

Edit

NFR 01	The system shall respond to search requests within 3 seconds.	Non-Functional	High
NFR-02	User data shall be encrypted during transmission and storage.	Non-Functional	High
NFR-03	The system shall support at least 1,000 concurrent users.	Non-Functional	Medium

3. User Stories

- As a student, I want to log into the system securely so that my personal library information is protected.
- As a student, I want to search for books by title, author, ISBN, or category so that I can quickly find the books I need.
- As a student, I want to check whether a book is available so that I know if I can borrow it.
- As a student, I want to borrow available books so that I can use them for my studies.
- As a student, I want to return borrowed books so that my borrowing records remain accurate.
- As a student, I want to view my borrowing history so that I can keep track of previously borrowed books.
- As a student, I want to receive due date reminders so that I can return books

Ask anything

ChatGPT can make mistakes. Check important info.

Smart Library System Requirement

chatgpt.com/c/6a40c27f-3840-83e8-a4c9-cb623b02c3b6

New chat

Search chats

Library

Projects

Scheduled

Apps

Codex

More

Recents

Smart Library System Requirement...

Mini Project 2 Help

Assignment Instructions Overview

ETL Pipeline for Dataset

BigGAN Image Generation Tips

SQL Query Translation

Copyable SQL Prompt

1% loss calculation

C++ Code for Locks

Incomplete Problem Statement

himikarshi

ChatGPT

Edit

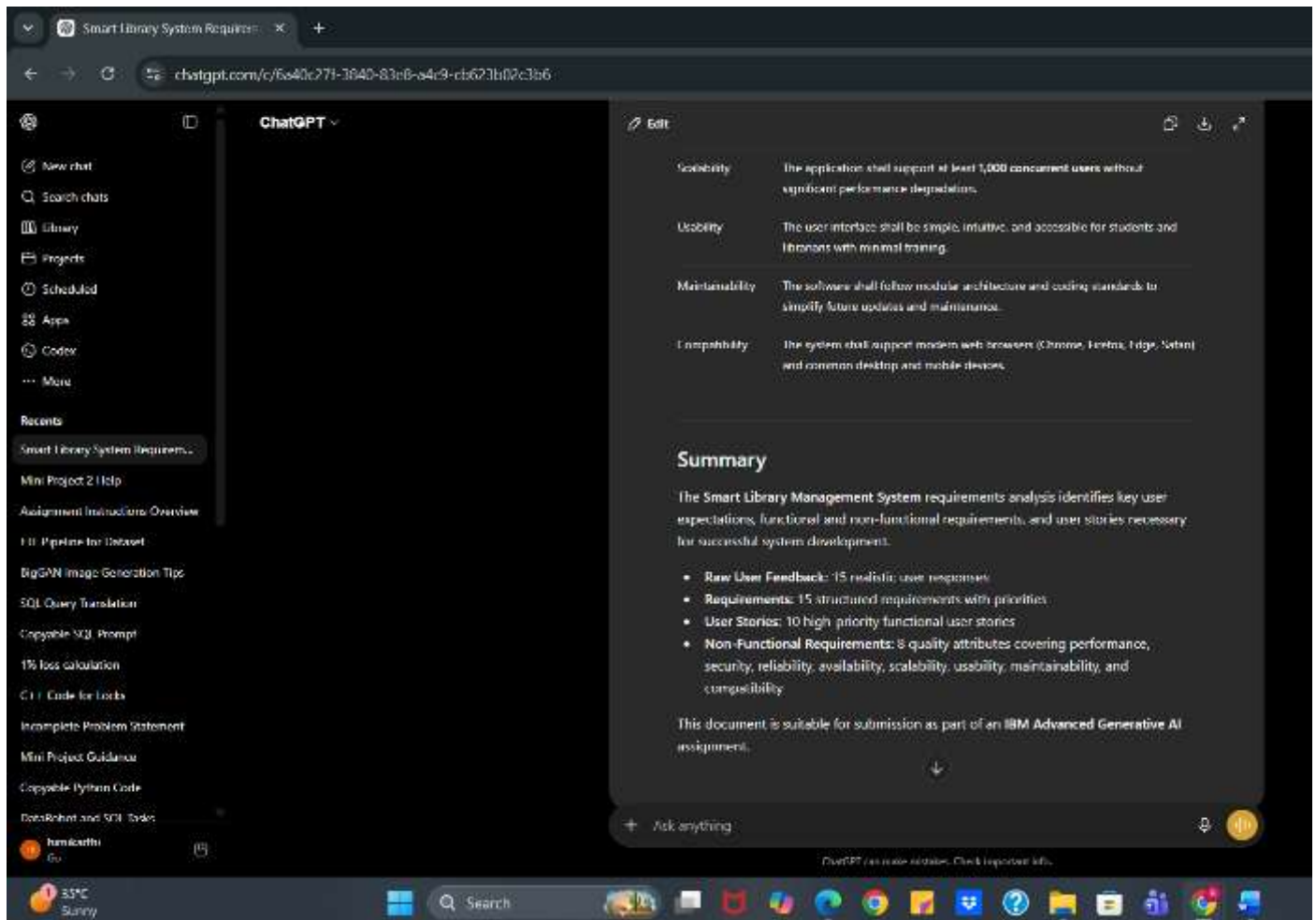
- As a student, I want to receive due date reminders so that I can return books on time and avoid fines.
- As a librarian, I want to manage book records so that the library catalogue remains accurate and up to date.
- As a librarian, I want to manage member accounts so that only authorized users can borrow books.
- As a librarian, I want to record issue and return transactions so that borrowing records are always accurate.

4. Non-Functional Requirements

Category	Non-Functional Requirement
Performance	The system shall respond to search requests within 3 seconds under normal load.
Security	All user passwords shall be encrypted, and communication shall use secure HTTPS protocols.
Reliability	The system shall maintain accurate transaction records with no data loss during normal operation.
Availability	The system shall maintain at least 99.9% uptime excluding scheduled maintenance.
Scalability	The application shall support at least 1,000 concurrent users without

Ask anything

ChatGPT can make mistakes. Check important info.



PHASE 2

Wireframe Design

Tool Used: Visily – app.visily.ai

Prompt:

“Design a Smart Library Management System web application with three screens.

Screen 1: Home Page

- Logo: Smart Library Management System
- Navigation bar: Home, Books, Dashboard, Contact
- Search bar for books
- Featured Books section with book cards
- Login and Register buttons
- Footer with contact information

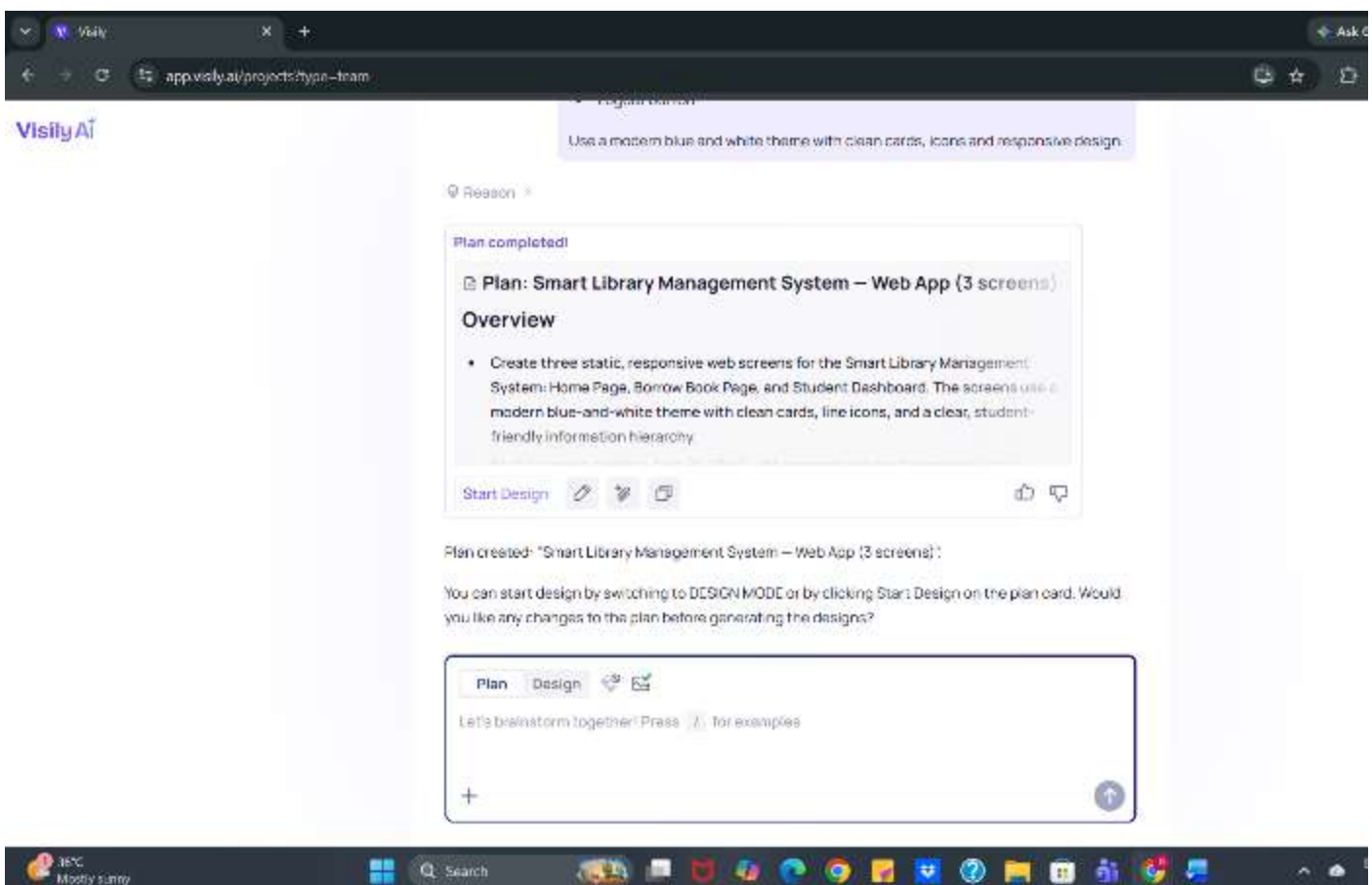
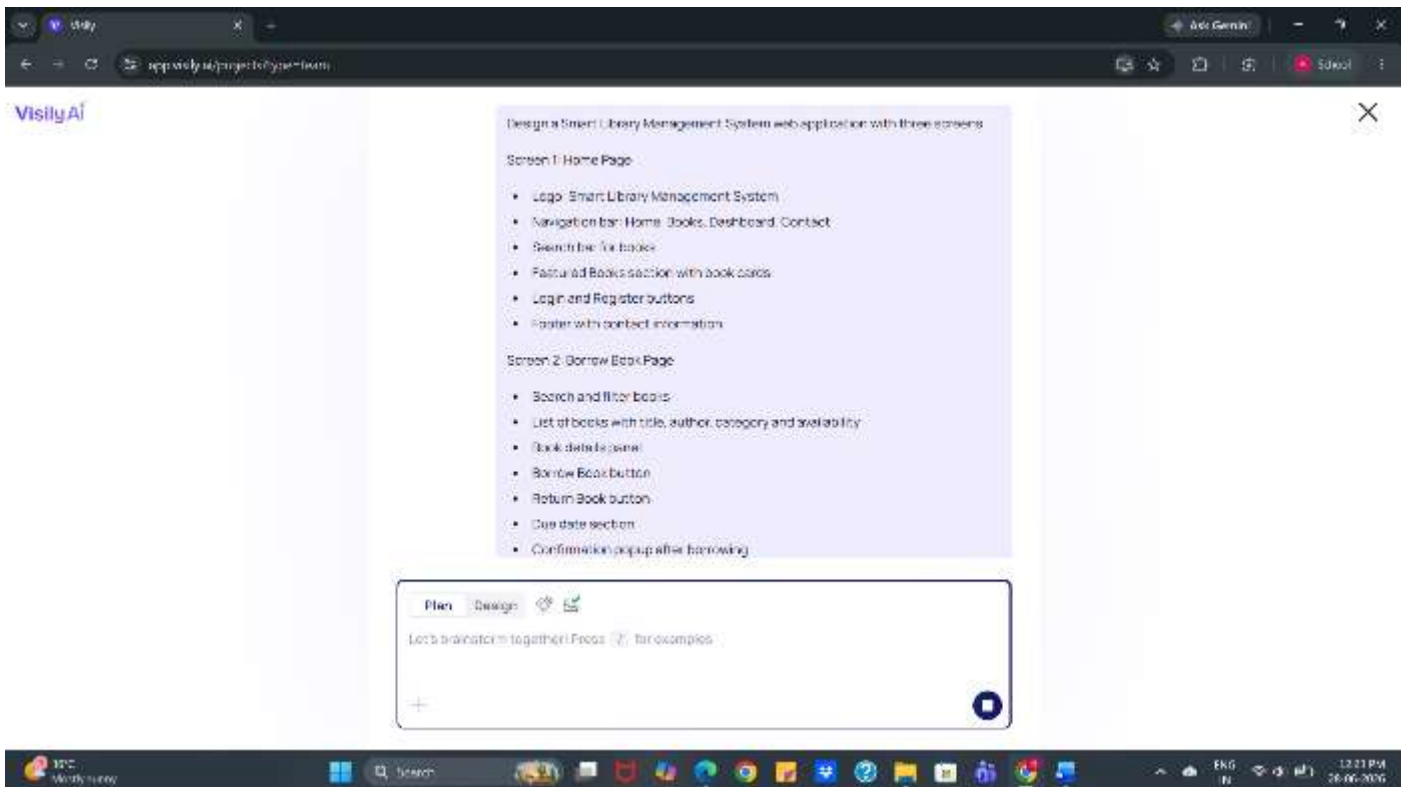
Screen 2: Borrow Book Page

- *Search and filter books*
- *List of books with title, author, category and availability*
- *Book details panel*
- *Borrow Book button*
- *Return Book button*
- *Due date section*
- *Confirmation popup after borrowing*

Screen 3: Student Dashboard

- *Student profile*
- *Books currently borrowed*
- *Borrow history*
- *Due date reminders*
- *Notifications*
- *Summary cards showing Active Books, Returned Books and Overdue Books*
- *Logout button*

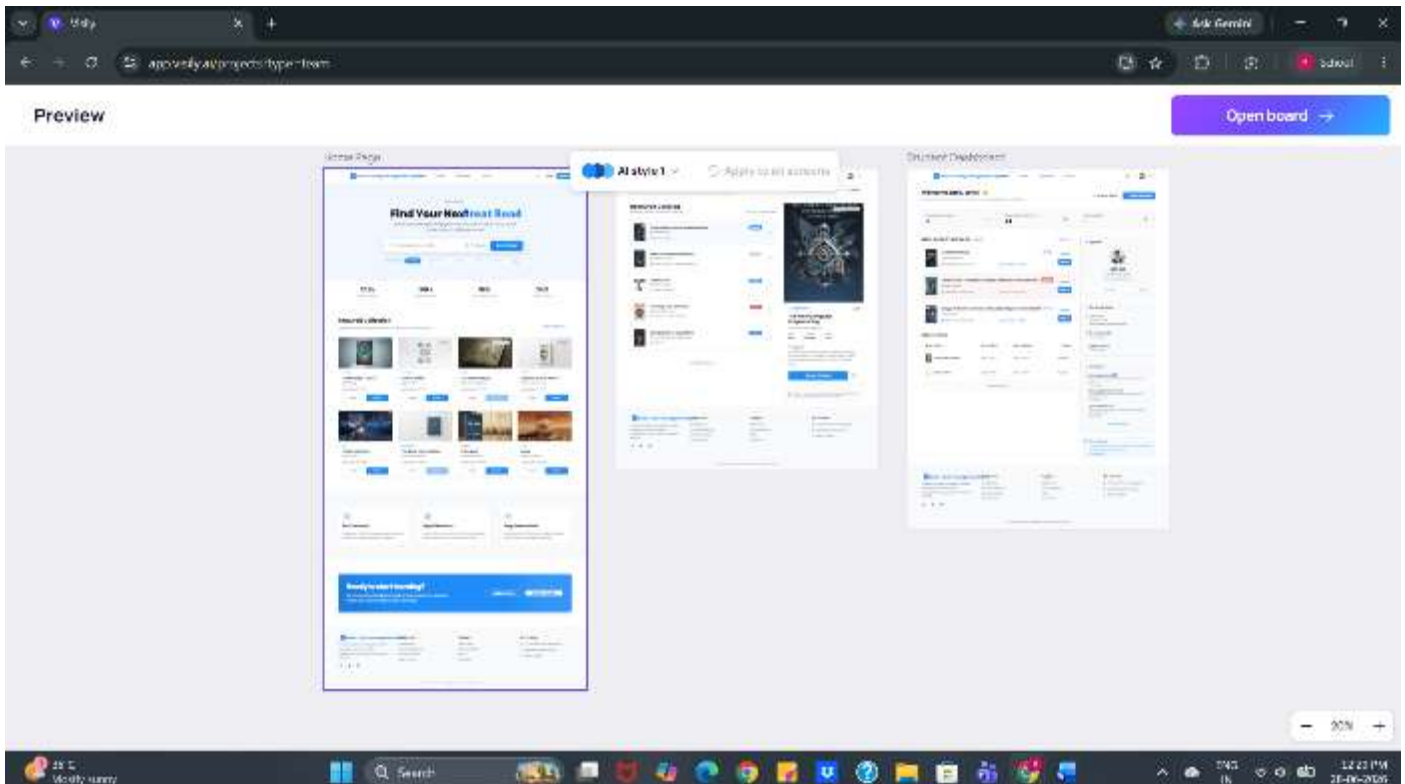
Use a modern blue and white theme with clean cards, icons and responsive design.”



Plan: Smart Library Management System – Web App (3 screens)

Execute plan: Smart Library Management System – Web App (3 screens)

I'll generate three WebApp screens (desktop 1440px): Home Page, Borrow Book Page, and Student Dashboard, using the plan you provided. I'll now create the screens.



Homepage



Smart Catalogue System

Find Your Next Great Read

Access over 50,000 digital and physical resources from our smart library. Search by title, author, or ISBN to get started.



Search by title, author, or ISBN...

All Categories

Search Catalog

Quick Search

All Books

Computer Science

Mathematics

Medicine

Science

Language

12.5k

ACTIVE STUDENTS

50k+

BOOKS AVAILABLE

98%

USER SATISFACTION

24/7

DIGITAL ACCESS

Featured Collection

Hand-picked recommendations based on current trends and curricula.

[Explore All Books](#)

Fiction

The Midnight Library

by Matt Haig

★★★★★ (10 reviews)

Details

Borrow



Self-Help

Atomic Habits

by James Clear

★★★★★ (10 reviews)

Details

Borrow



Fiction

The Great Gatsby

by F. Scott Fitzgerald

★★★★★ (10 reviews)

Details

Borrow



Non-Fiction

Sapiens: A Brief History

by Yu. N. Harari

★★★★★ (10 reviews)

Details

Borrow



Sci-Fi

Project Hail Mary

by Andy Weir

★★★★★ (10 reviews)

Details

Borrow



Psychology

Thinking, Fast and Slow

by Daniel Kahneman

★★★★★ (10 reviews)

Details

Borrow



Non-Fiction

Educated

by Tara Westover

★★★★★ (10 reviews)

Details

Borrow



Sci-Fi

Dune

by Frank Herbert

★★★★★ (10 reviews)

Details

Borrow



Fast Renewals

Running out of time? Renew your books online with a single click from your personal dashboard.



Digital Reserves

Access PDF versions and e-books for core textbooks, available instantly for authorized students.



Easy Reservations

Book unavailable? Place a hold and get notified via email as soon as it's returned to the rack.

Ready to start learning?

Join thousands of students and gain access to premium resources. Create your account today to start borrowing.

[Register Now](#)[Student Login](#)

Smart Library Management by **Book Link**

Empowering students through accessible knowledge and smart resource management, your gateway to a world of learning.



Search Books
Student Dashboard
Membership Rules
Book Lists

Support

Help Center
Privacy Policy
FAQs
Contact Us

Get in Touch

123 University Ave, Library Hall
Email: support@booklink.edu
Phone: (555) 123-4567

Borrow page

Smart Library Management System

Books

Dashboard

Contact

🔍

👤

🔙

🔍 Search by title, author, or ISBN...

AllAvailableBorrowed

Filter By

Resource Catalog

Browsing 1,245 titles in the library collection

Sort By: NEWEST FIRST

The Art of Computer Programming

By Donald Knuth

Computer Science

Available

Deep Learning with Python

By François Chollet

Artificial Intelligence

Due: 2024-05-15

Borrowed

Clean Code

By Robert C. Martin

Software Engineering

Available

Thinking, Fast and Slow

By Daniel Kahneman

Psychology

Due: 2024-04-20

Overdue

Introduction to Algorithms

By Cormen, Leiserson, Rivest, Stein

Algorithms

Available

View More Results

THE ART OF COMPUTER PROGRAMMING

DONALD E. KNUTH

Computer Science

★ 4.9

The Art of Computer Programming

Written by Donald Knuth

ISBN

672

STATUS

Available

COPIES

3/5

Synopsis

A comprehensive monograph which is widely considered the definitive work on computer programming. It covers fundamental algorithms and their analysis in extreme detail.

Borrow This Book

🔖

🕒 Standard loan period is 14 days. You can renew this book once if there are no pending holds from other students.

Smart Library Management System

Quick Links

Empowering students through accessible knowledge and smart resource management. Your gateway to a world of learning.

📖

👤

🔖

Search Books

Student Dashboard

Membership Rules

Library Events

Support

Help Center

Borrowing Policy

FAQs

Contact Us

Get In Touch

📍 123 University Ave. Library Hall

✉️ support@smartlibrary.edu

☎️ (555) 123-4567

© 2024 Smart Library Management System. All rights reserved.

Made with Vercel

Student dashboard

Welcome back, John! 📖

You have 1 book overdue. Please return it as soon as possible to avoid fines.

[View Full History](#)[Search New Books](#)

Active Borrowed Books

3



Books Returned (All Time)

24



Overdue Books

1



Books Currently Borrowed

[3 items](#)[View All](#)

The Great Gatsby

F. Scott Fitzgerald

Borrowed: Oct 15, 2023

Due: Nov 12, 2023

[On Loan](#)[Renew](#)[Return](#)

Clean Code: A Handbook of Agile Software Craftsmanship

Robert C. Martin

Borrowed: Oct 02, 2023

Due: Oct 28, 2023

[On Loan](#)[Renew](#)[Return](#)

Design Patterns: Elements of Reusable Object-Oriented Software

Erich Gamma

Borrowed: Oct 20, 2023

Due: Nov 17, 2023

[On Loan](#)[Renew](#)[Return](#)

Recent History

Book Details

Borrow Date

Returned Date

Status



To Kill a Mockingbird

Sep 01, 2023

Sep 25, 2023

Returned



Atomic Habits

Aug 15, 2023

Sep 12, 2023

Returned

[Show more history...](#)

My Profile

**John Doe**

j.doe@university.edu

ID: 12345678901234567890

[Edit Profile](#)[Logout](#)

Due Date Reminders

Clean Code:
Overdue by 3 days.
Please return it, Central Wing Level 2.

The Great Gatsby
Due in 14 days

Design Patterns
Due in 14 days

Notifications

New Book Arrival 📖
"Atomic Habits" by James Clear is now available in the library.
2 weeks ago

Borrowing Window Extended
The library has extended holiday borrowing limits to 5 books per student.
(2023-2024)

System Maintenance
Library portal will be offline for maintenance on Sunday from 2 AM to 4 AM.
Oct 22, 2023

[View All Notifications](#)

Did you know?
You can extend your borrowing period up to 2 times unless if no one else has reserved the book.
[Learn more](#) 📖

Smart Library Management System

Empowering students through accessible knowledge and smart resource management. Your gateway to a world of learning.



Quick Links

[Search Books](#)
[Student Dashboard](#)
[Membership Rules](#)
[Library Events](#)

Support

[Help Center](#)
[Borrowing Policy](#)
[FAQs](#)
[Contact Us](#)

Get In Touch

📍 123 University Ave. Library Hall
✉ support@smartlibrary.edu
☎ (555) 123-4567

A short explanation of how the three screens connect and what each screen does

The Smart Library Management System consists of three connected screens. The Home Screen allows users to search books and access the system. The Borrow Book Screen enables students to search, borrow, and return books while displaying due dates. After borrowing, users can access the Dashboard to view their profile, borrowing history, active books, due date reminders, and notifications. These screens provide a simple and user-friendly workflow.

PHASE 3

Architecture and Database design

Tool Used: Eraser.io

Architecture Design

Prompt:

"Create a complete system architecture diagram for a Smart Library Management System.

The architecture should include:

Frontend:

- Web Application*
- Home Page*
- Search Books*
- Borrow / Return Books*
- Student Dashboard*

Backend:

- Python Application*
- Authentication Module*
- Book Management Module*
- Borrow & Return Module*
- Notification Module*

Database:

- SQLite Database*

Database Tables:

- *Students*
- *Librarians*
- *Books*
- *BorrowRecords*

External Services:

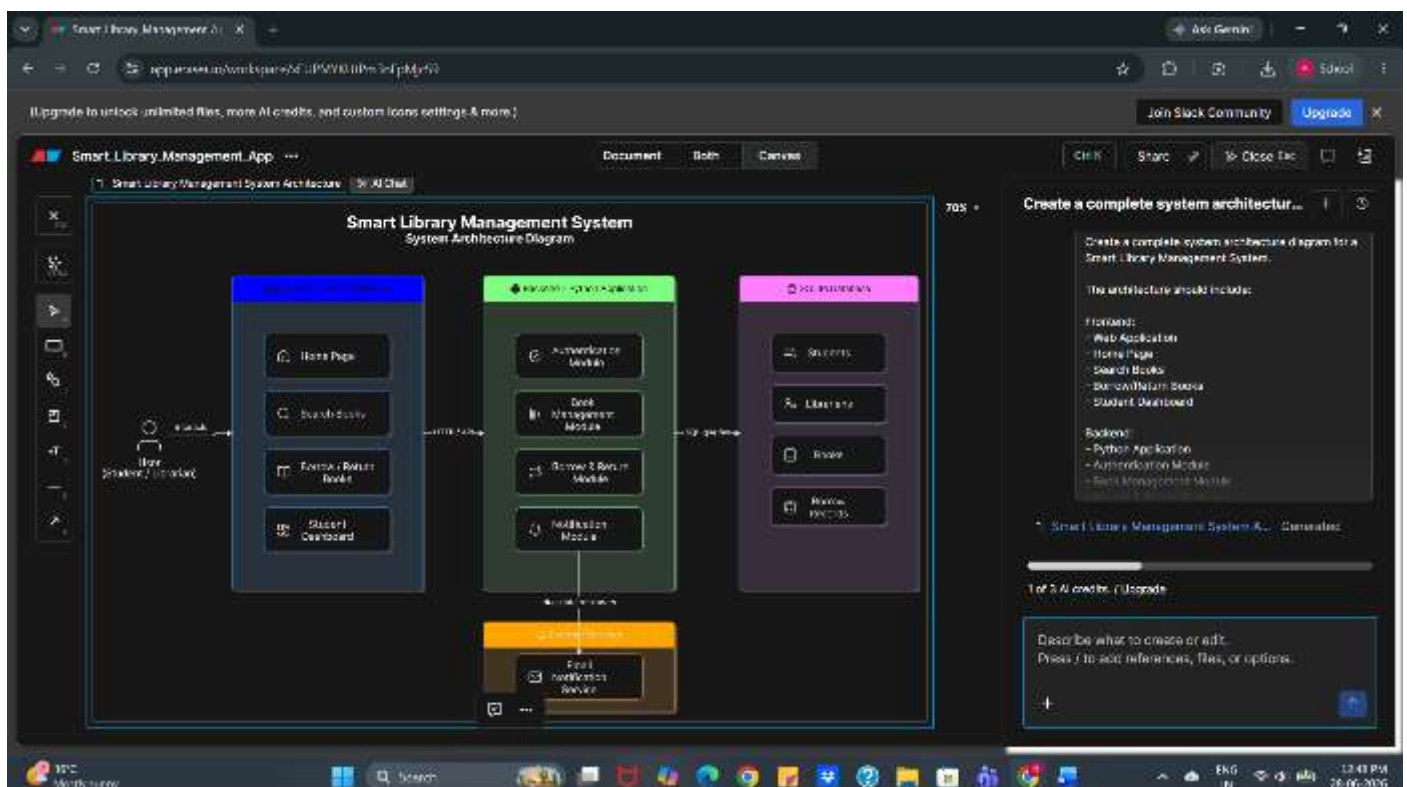
- *Email Notification Service*

Show arrows indicating:

User → Frontend → Backend → SQLite Database

Backend also connects to the Email Notification Service for due-date reminders.

Create a clean software architecture diagram suitable for an academic assignment.”



ER Diagram

Prompt:

“Generate an Entity Relationship Diagram (ERD) for a Smart Library Management System.

Include the following tables and fields.

Students

- *student_id (Primary Key)*
- *name*
- *email*
- *phone*
- *department*

Librarians

- *librarian_id (Primary Key)*
- *name*
- *email*

Books

- *book_id (Primary Key)*
- *title*
- *author*
- *category*
- *isbn*
- *available_copies*

BorrowRecords

- *borrow_id (Primary Key)*

- *student_id* (Foreign Key)
- *book_id* (Foreign Key)
- *borrow_date*
- *due_date*
- *return_date*
- *status*

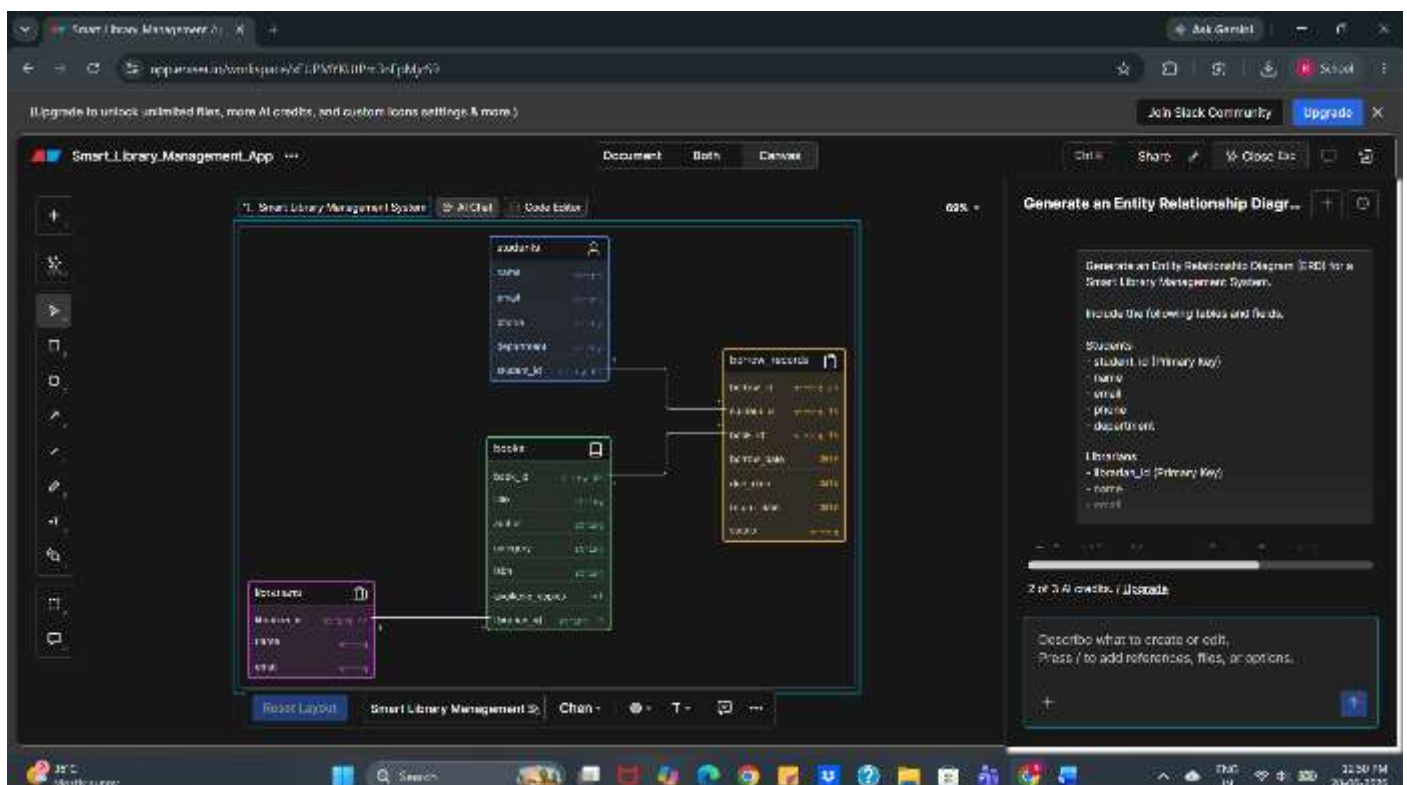
Relationships:

One Student can borrow many Books through BorrowRecords.

One Book can appear in many BorrowRecords.

One Librarian manages many Books.

Display all primary keys, foreign keys, and relationship lines."



SQL Create Table Statements

```
CREATE TABLE Students (  
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    email TEXT UNIQUE NOT NULL,  
    phone TEXT,  
    department TEXT  
);
```

```
CREATE TABLE Librarians (  
    librarian_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    email TEXT UNIQUE NOT NULL  
);
```

```
CREATE TABLE Books (  
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    author TEXT NOT NULL,  
    category TEXT,  
    isbn TEXT UNIQUE,  
    available_copies INTEGER CHECK (available_copies >= 0)  
);
```

```
CREATE TABLE BorrowRecords (  
    borrow_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    student_id INTEGER,  
    book_id INTEGER,
```

```
borrow_date DATE,  
  
due_date DATE,  
  
return_date DATE,  
  
status TEXT CHECK (status IN ('Borrowed','Returned')),  
  
FOREIGN KEY(student_id) REFERENCES Students(student_id),  
  
FOREIGN KEY(book_id) REFERENCES Books(book_id)  
  
);
```

Database Explanation:

-  **Students:** Stores student details such as ID, name, email, phone number, and department.
-  **Librarians:** Stores librarian information responsible for managing books.
-  **Books:** Stores book details including title, author, category, ISBN, and available copies.
-  **BorrowRecords:** Tracks every borrowing transaction, including the student, book, borrow date, due date, return date, and current status. It connects the Students and Books tables through foreign keys.

PHASE 4

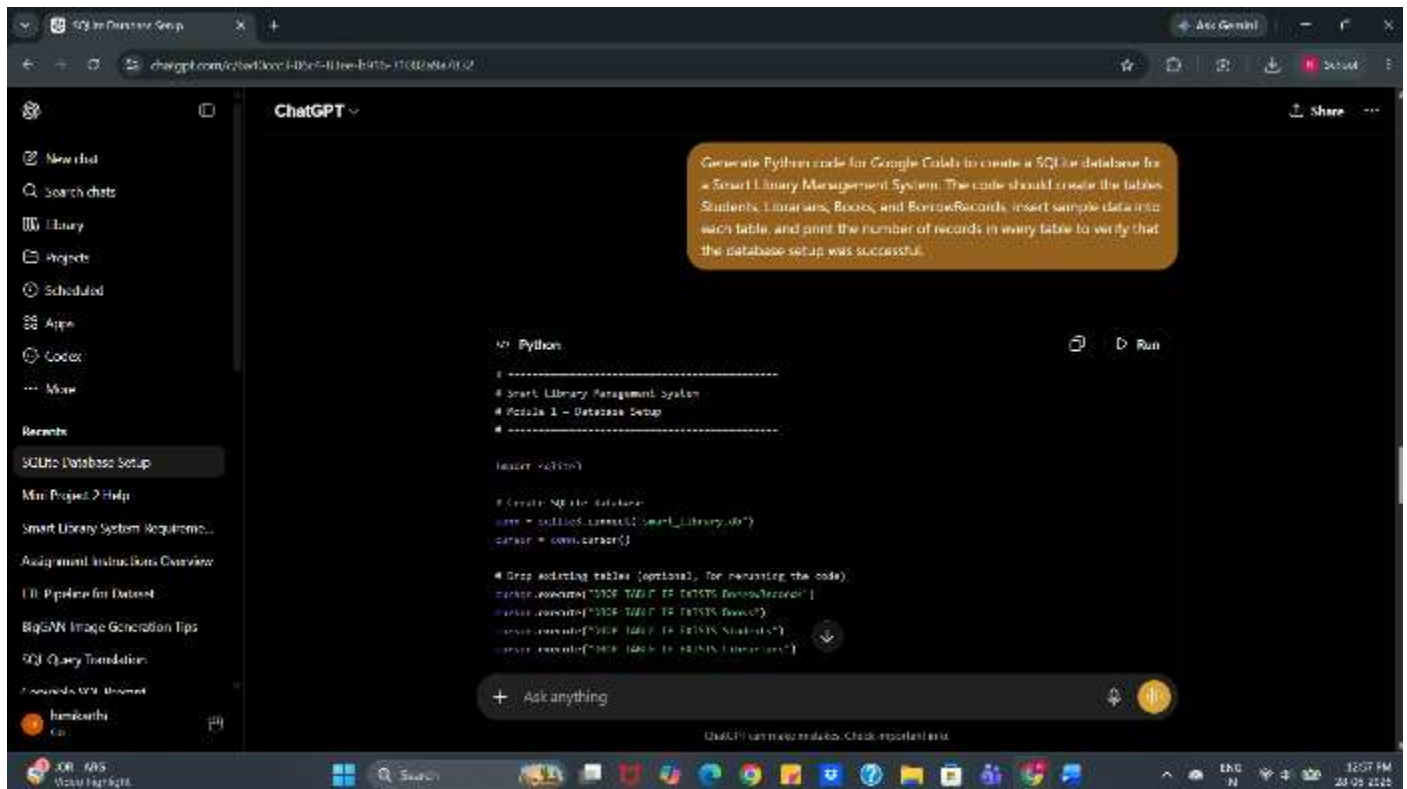
Development

Tool: Google Colab

Module 1 – Database Setup

Prompt:

“Generate Python code for Google Colab to create a SQLite database for a Smart Library Management System. The code should create the tables Students, Librarians, Books, and BorrowRecords, insert sample data into each table, and print the number of records in every table to verify that the database setup was successful.”



CODE

[1]
✓ Os

```
import sqlite3

# Connect to SQLite database
conn = sqlite3.connect("library.db")
cursor = conn.cursor()

# -----
# Create Tables
# -----

cursor.execute("""
CREATE TABLE IF NOT EXISTS Students (
    student_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT,
    department TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Librarians (
    librarian_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Books (
    book_id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    author TEXT,
    category TEXT,
    available_copies INTEGER
)
""")
```

```

▶ cursor.execute("""
CREATE TABLE IF NOT EXISTS BorrowRecords (
    borrow_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    book_id INTEGER,
    borrow_date TEXT,
    due_date TEXT,
    status TEXT,
    FOREIGN KEY(student_id) REFERENCES Students(student_id),
    FOREIGN KEY(book_id) REFERENCES Books(book_id)
)
""")

# -----
# Insert Sample Data
# -----

students = [
    (1, "Rahul", "rahul@gmail.com", "CSE"),
    (2, "Priya", "priya@gmail.com", "ECE"),
    (3, "Arun", "arun@gmail.com", "IT")
]

librarians = [
    (1, "Meena", "meena@library.com"),
    (2, "Ravi", "ravi@library.com")
]

books = [
    (1, "Python Programming", "John Smith", "Programming", 5),
    (2, "Database Systems", "Navathe", "Database", 4),
    (3, "Artificial Intelligence", "Stuart Russell", "AI", 3),
    (4, "Machine Learning", "Tom Mitchell", "AI", 2),
    (5, "Data Structures", "Mark Allen", "Programming", 6)
]

borrow_records = [
    (1, 1, 2, "2026-06-20", "2026-07-05", "Borrowed"),
    (2, 2, 1, "2026-06-18", "2026-07-03", "Returned"),
    (3, 3, 3, "2026-06-25", "2026-07-10", "Borrowed")
]

```

```

cursor.executemany("INSERT OR REPLACE INTO Students VALUES (?, ?, ?, ?)", students)
cursor.executemany("INSERT OR REPLACE INTO Librarians VALUES (?, ?, ?)", librarians)
cursor.executemany("INSERT OR REPLACE INTO Books VALUES (?, ?, ?, ?, ?)", books)
cursor.executemany("INSERT OR REPLACE INTO BorrowRecords VALUES (?, ?, ?, ?, ?, ?)", borrow_records)

conn.commit()

# -----
# Verify Record Count
# -----

tables = ["Students", "Librarians", "Books", "BorrowRecords"]

print("=" * 45)
print("SMART LIBRARY MANAGEMENT SYSTEM")
print("Database Setup Verification")
print("=" * 45)

for table in tables:
    cursor.execute(f"SELECT COUNT(*) FROM {table}")
    count = cursor.fetchone()[0]
    print(f"{table}: {count} records")

print("\nDatabase setup completed successfully!")

conn.close()

```

OUTPUT

```
... =====
SMART LIBRARY MANAGEMENT SYSTEM
Database Setup Verification
=====
Students: 3 records
Librarians: 2 records
Books: 5 records
BorrowRecords: 3 records

Database setup completed successfully!
```

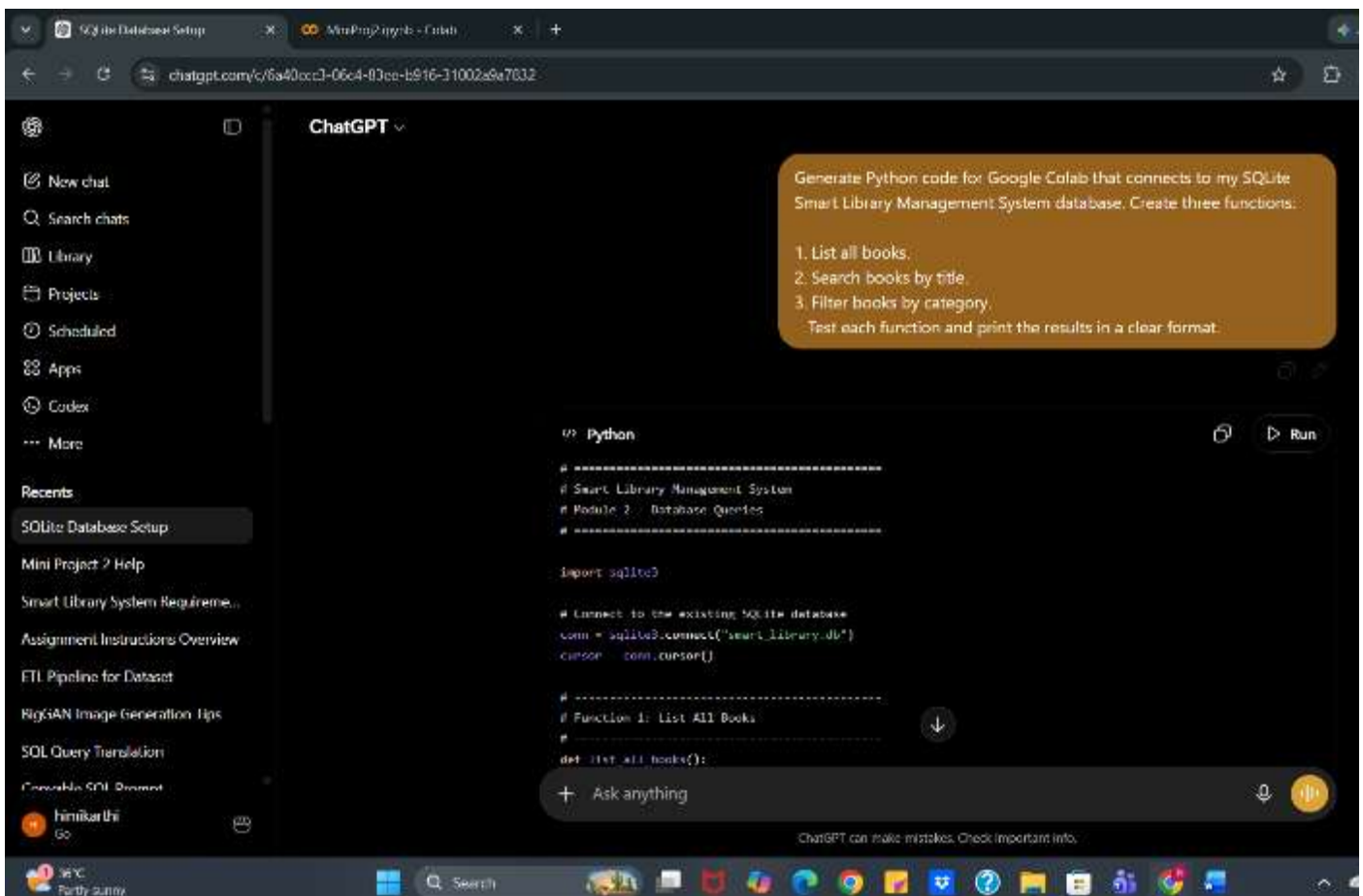
Module 2 – Core Data Functions

Prompt:

“Generate Python code for Google Colab that connects to my SQLite Smart Library Management System database. Create three functions:

- 1. List all books.*
- 2. Search books by title.*
- 3. Filter books by category.*

Test each function and print the results in a clear format.”



CODE



```
import sqlite3

# Connect to existing database
conn = sqlite3.connect("library.db")
cursor = conn.cursor()

# -----
# Function 1: List All Books
# -----
def list_books():
    print("\n===== ALL BOOKS =====")
    cursor.execute("SELECT * FROM Books")
    books = cursor.fetchall()

    for book in books:
        print(book)

# -----
# Function 2: Search Book
# -----
def search_book(title):

    print(f"\nSearching for '{title}'")

    cursor.execute("""
    SELECT * FROM Books
    WHERE title LIKE ?
    """, ('%'+title+'%',))

    books = cursor.fetchall()

    if books:
        for book in books:
            print(book)
    else:
        print("No book found.")
```

```

# -----
# Function 3: Filter Books
# -----
def filter_books(category):

    print(f"\nBooks in Category: {category}")

    cursor.execute("""
    SELECT * FROM Books
    WHERE category=?
    """, (category,))

    books=cursor.fetchall()

    if books:
        for book in books:
            print(book)
    else:
        print("No books found.")

# -----
# Testing Functions
# -----

list_books()

search_book("Python")

filter_books("AI")

conn.close()

```

OUTPUT

```

...
===== ALL BOOKS =====
(1, 'Python Programming', 'John Smith', 'Programming', 5)
(2, 'Database Systems', 'Navathe', 'Database', 4)
(3, 'Artificial Intelligence', 'Stuart Russell', 'AI', 3)
(4, 'Machine Learning', 'Tom Mitchell', 'AI', 2)
(5, 'Data Structures', 'Mark Allen', 'Programming', 6)

Searching for 'Python'
(1, 'Python Programming', 'John Smith', 'Programming', 5)

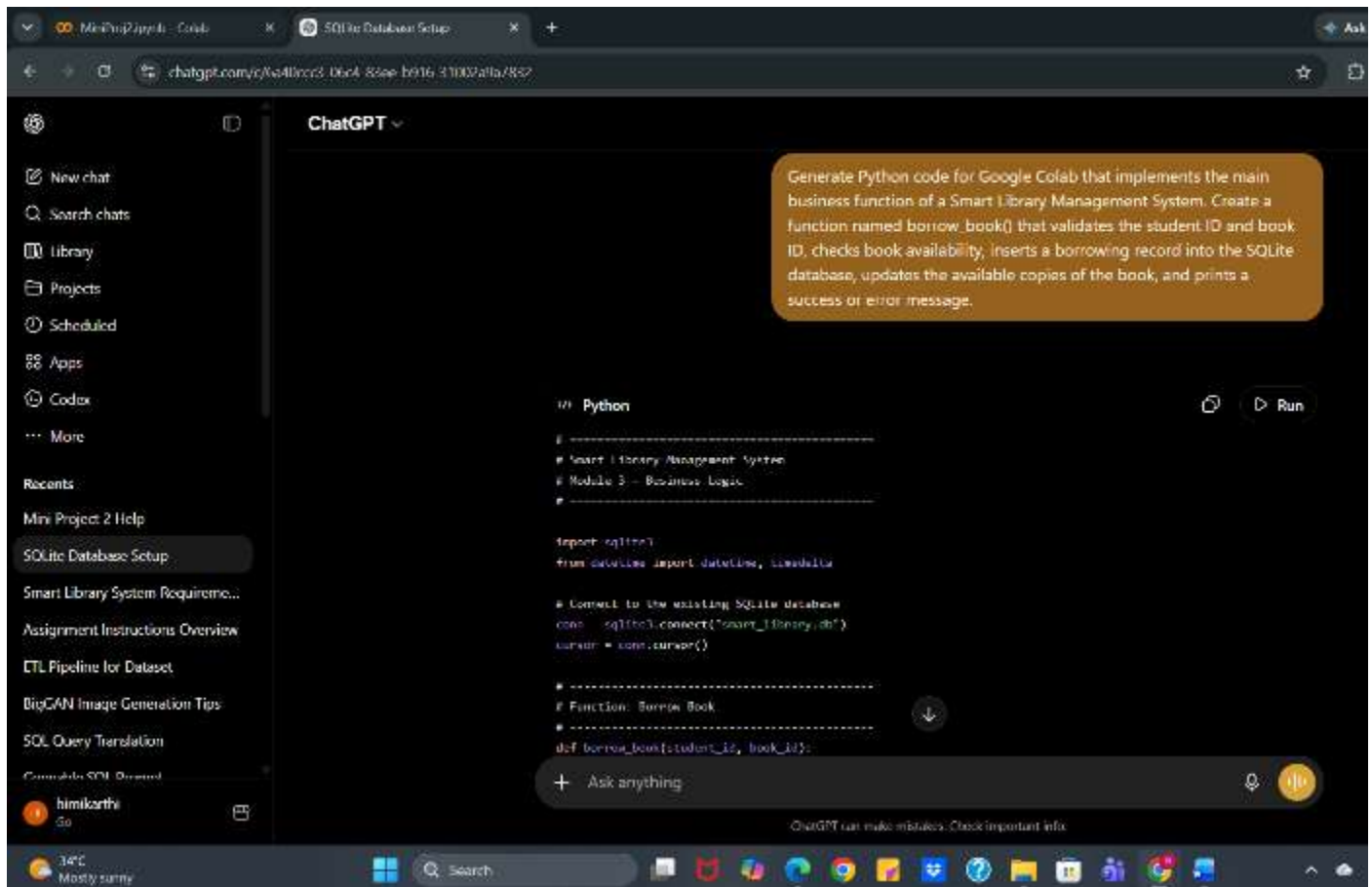
Books in Category: AI
(3, 'Artificial Intelligence', 'Stuart Russell', 'AI', 3)
(4, 'Machine Learning', 'Tom Mitchell', 'AI', 2)

```

Module 3 – Main Business Function

Prompt:

“Generate Python code for Google Colab that implements the main business function of a Smart Library Management System. Create a function named `borrow_book()` that validates the student ID and book ID, checks book availability, inserts a borrowing record into the SQLite database, updates the available copies of the book, and prints a success or error message.”



The screenshot shows the ChatGPT web interface. The prompt is displayed in a yellow box at the top right. The generated Python code is shown in a dark-themed editor with a left sidebar containing navigation options like 'New chat', 'Search chats', 'Library', 'Projects', 'Scheduled', 'Apps', 'Codex', and 'More'. The code is for a Smart Library Management System, specifically Module 3 - Business Logic. It includes imports for sqlite3, datetime, and timedelta, and a function named borrow_book that takes student_id and book_id as arguments. The code is ready to be run, as indicated by the 'Run' button in the top right corner of the code editor.

```
1/1 Python Run  
  
# Smart Library Management System  
# Module 3 - Business Logic  
#  
  
import sqlite3  
from datetime import datetime, timedelta  
  
# Connect to the existing SQLite database  
conn = sqlite3.connect("smart_library.db")  
cursor = conn.cursor()  
  
# Function: Borrow Book  
#  
def borrow_book(student_id, book_id):  
    # Validate student ID and book ID  
    if not student_id or not book_id:  
        print("Invalid student ID or book ID")  
        return  
    # Check book availability  
    cursor.execute("SELECT available_copies FROM books WHERE book_id = ?", (book_id,))  
    row = cursor.fetchone()  
    if not row or row[0] == 0:  
        print("Book is not available")  
        return  
    # Insert borrowing record  
    cursor.execute("INSERT INTO borrowings (student_id, book_id, borrow_date) VALUES (?, ?, ?)",  
                  (student_id, book_id, datetime.now()))  
    # Update available copies  
    cursor.execute("UPDATE books SET available_copies = available_copies - 1 WHERE book_id = ?", (book_id,))  
    conn.commit()  
    print("Book borrowed successfully")  
    # Close the database connection  
    conn.close()
```

CODE



```
import sqlite3
from datetime import datetime, timedelta

# Connect to database
conn = sqlite3.connect("library.db")
cursor = conn.cursor()

# -----
# Borrow Book Function
# -----
def borrow_book(student_id, book_id):

    # Check if student exists
    cursor.execute("SELECT * FROM Students WHERE student_id=?", (student_id,))
    student = cursor.fetchone()

    if not student:
        print("✗ Student ID not found.")
        return

    # Check if book exists
    cursor.execute("SELECT * FROM Books WHERE book_id=?", (book_id,))
    book = cursor.fetchone()

    if not book:
        print("✗ Book ID not found.")
        return

    title = book[1]
    copies = book[4]

    # Check availability
    if copies <= 0:
        print(f"✗ '{title}' is currently unavailable.")
        return

    # Dates
    borrow_date = datetime.today().strftime("%Y-%m-%d")
    due_date = (datetime.today() + timedelta(days=14)).strftime("%Y-%m-%d")
```



```
# Generate Borrow ID
cursor.execute("SELECT MAX(borrow_id) FROM BorrowRecords")
result = cursor.fetchone()[0]

if result is None:
    borrow_id = 1
else:
    borrow_id = result + 1

# Insert Borrow Record
cursor.execute("""
INSERT INTO BorrowRecords
VALUES (?, ?, ?, ?, ?, ?)
""",
(
    borrow_id,
    student_id,
    book_id,
    borrow_date,
    due_date,
    "Borrowed"
))

# Update available copies
cursor.execute("""
UPDATE Books
SET available_copies = available_copies - 1
WHERE book_id=?
""",(book_id,))

conn.commit()

print("=====")
print("Book Borrowed Successfully!")
print("=====")
print("Student ID :", student_id)
print("Book :", title)
print("Borrow Date :", borrow_date)
print("Due Date :", due_date)
print("Status : Borrowed")
```

```
# -----
# Test the Function
# -----

borrow_book(1,4)

conn.close()
```

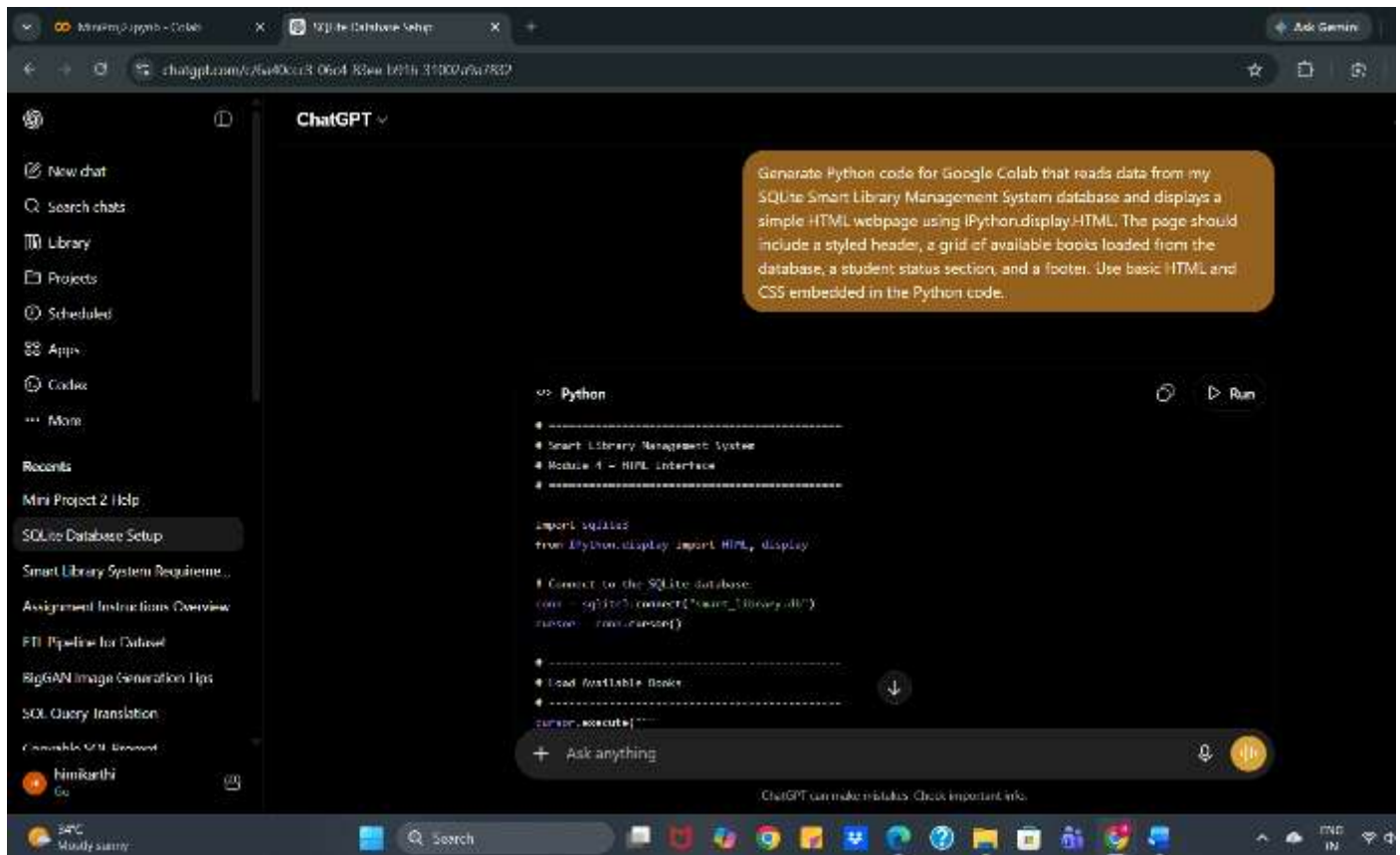
OUTPUT

```
... =====
Book Borrowed Successfully!
=====
Student ID : 1
Book : Machine Learning
Borrow Date : 2026-06-28
Due Date : 2026-07-12
Status : Borrowed
```

Module 4 – HTML Frontend

Prompt:

“Generate Python code for Google Colab that reads data from my SQLite Smart Library Management System database and displays a simple HTML webpage using IPython.display.HTML. The page should include a styled header, a grid of available books loaded from the database, a student status section, and a footer. Use basic HTML and CSS embedded in the Python code.”



CODE

```

import sqlite3
from IPython.display import HTML, display

# Connect to SQLite database
conn = sqlite3.connect("library.db")
cursor = conn.cursor()

# -----
# Fetch Books
# -----
cursor.execute("""
SELECT title, author, category, available_copies
FROM Books
""")
books = cursor.fetchall()

# -----
# Fetch Student Status
# -----
cursor.execute("""
SELECT COUNT(*) FROM Students
""")
students = cursor.fetchone()[0]

cursor.execute("""
SELECT COUNT(*) FROM BorrowRecords
WHERE status='Borrowed'
""")
borrowed = cursor.fetchone()[0]

```

```

# -----
# Generate Book Cards
# -----
cards = ""

for title, author, category, copies in books:
    cards += f"""
    <div class="card">
        <h3>{title}</h3>
        <p><strong>Author:</strong> {author}</p>
        <p><strong>Category:</strong> {category}</p>
        <p><strong>Available Copies:</strong> {copies}</p>
    </div>
    """

# -----
# HTML Page
# -----
html = f"""

<!DOCTYPE html>

<html>

<head>

<style>

body {{
    font-family: Arial, sans-serif;
    background: #f5f7fa;
    margin: 0;
}}

```

```
▶ header {{
  background: #1f4e79;
  color: white;
  text-align: center;
  padding: 25px;
}}

header h1 {{
  margin: 0;
}}

.container {{
  padding: 25px;
}}

.grid {{
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(230px,1fr));
  gap: 20px;
}}

.card {{
  background: white;
  border-radius: 12px;
  padding: 18px;
  box-shadow: 0 4px 10px rgba(0,0,0,0.15);
}}
```

```
▶ .card h3 {{
  color: #1f4e79;
  margin-bottom: 12px;
}}

.card p {{
  color: #333;
  margin: 6px 0;
}}

.status {{
  margin-top: 35px;
  background: #dff3ff;
  border-radius: 12px;
  padding: 20px;
}}

.status h2 {{
  color: #1f4e79;
}}
```



```

▶ .status p {{
    color: #222;
    font-size: 16px;
  }}

  footer {{
    margin-top: 35px;
    background: #1f4e79;
    color: white;
    text-align: center;
    padding: 15px;
    font-size: 15px;
  }}

</style>

</head>

<body>

<header>

<h1>📖 Smart Library Management System</h1>

<p>Google Colab HTML Frontend</p>

</header>

```

```

▶ <div class="container">

<h2 style="color:#1f4e79;">Available Books</h2>

<div class="grid">

{cards}

</div>

<div class="status">

<h2>Student Status</h2>

<p><strong>Total Students:</strong> {students}</p>

<p><strong>Books Currently Borrowed:</strong> {borrowed}</p>

<p><strong>Status:</strong> ✅ Library System Running Successfully</p>

</div>

</div>

<footer>

© 2026 Smart Library Management System | IBM Advanced Generative AI Assignment

</footer>

</body>

</html>

"""

display(HTML(html))

conn.close()

```

OUTPUT



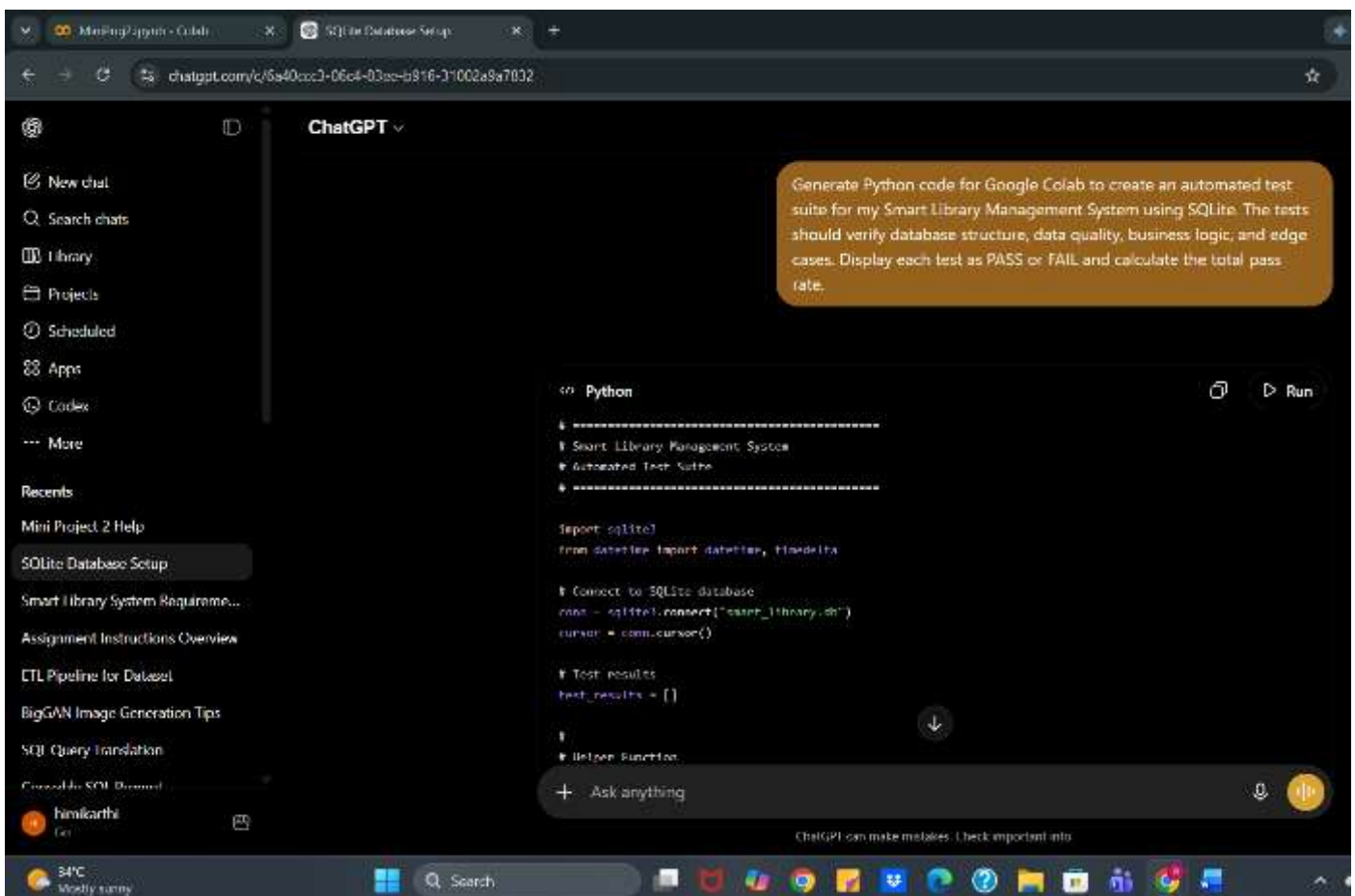
PHASE 5

Testing

Tool Used: Google Colab

Prompt:

“Generate Python code for Google Colab to create an automated test suite for my Smart Library Management System using SQLite. The tests should verify database structure, data quality, business logic, and edge cases. Display each test as PASS or FAIL and calculate the total pass rate.”



CODE



```
import sqlite3

conn = sqlite3.connect("library.db")
cursor = conn.cursor()

results = []

# -----
# Helper Function
# -----
def run_test(name, condition):
    if condition:
        results.append(("PASS", name))
    else:
        results.append(("FAIL", name))

# =====
# DATABASE STRUCTURE TESTS
# =====

tables = ["Students", "Librarians", "Books", "BorrowRecords"]

for table in tables:
    cursor.execute("""
        SELECT name
        FROM sqlite_master
        WHERE type='table' AND name=?
        """, (table,))
    run_test(f"Table '{table}' exists", cursor.fetchone() is not None)

# =====
# DATA QUALITY TESTS
# =====

# Negative available copies
cursor.execute("""
SELECT COUNT(*)
FROM Books
WHERE available_copies < 0
""")
```

```

run_test(
    "No negative available copies",
    cursor.fetchone()[0] == 0
)

# Null student names
cursor.execute("""
SELECT COUNT(*)
FROM Students
WHERE name IS NULL
""")
run_test(
    "No NULL student names",
    cursor.fetchone()[0] == 0
)

# Empty book titles
cursor.execute("""
SELECT COUNT(*)
FROM Books
WHERE title IS NULL OR title=''
""")
run_test(
    "No empty book titles",
    cursor.fetchone()[0] == 0
)

# =====
# BUSINESS LOGIC TESTS
# =====

# Borrow records exist
cursor.execute("""
SELECT COUNT(*)
FROM BorrowRecords
""")
run_test(
    "Borrow records created",
    cursor.fetchone()[0] > 0
)

```

```

▶ # Available copies never negative
cursor.execute("""
SELECT MIN(available_copies)
FROM Books
""")
run_test(
    "Book stock updated correctly",
    cursor.fetchone()[0] >= 0
)

# =====
# EDGE CASE TESTS
# =====

# Search for non-existing book
cursor.execute("""
SELECT *
FROM Books
WHERE title='Unknown Book'
""")
run_test(
    "Search handles missing books",
    len(cursor.fetchall()) == 0
)

# Duplicate student email
cursor.execute("""
SELECT email,
COUNT(*)
FROM Students
GROUP BY email
HAVING COUNT(*)>1
""")
run_test(
    "No duplicate student emails",
    len(cursor.fetchall()) == 0
)

```

```

# =====
# TEST REPORT
# =====

print("="*60)
print(" SMART LIBRARY MANAGEMENT SYSTEM")
print(" AUTOMATED TEST REPORT")
print("="*60)

passed = 0

for status, test in results:
    icon = "✅" if status=="PASS" else "❌"
    print(f"{icon} {test} : {status}")
    if status=="PASS":
        passed += 1

total = len(results)
rate = (passed/total)*100

print("\n"+"="*60)
print(f"Total Tests : {total}")
print(f"Passed      : {passed}")
print(f"Failed       : {total-passed}")
print(f"Pass Rate    : {rate:.2f}%")
print("="*60)

conn.close()

```

OUTPUT

```
*** =====
SMART LIBRARY MANAGEMENT SYSTEM
AUTOMATED TEST REPORT
=====
✅ Table 'Students' exists : PASS
✅ Table 'Librarians' exists : PASS
✅ Table 'Books' exists : PASS
✅ Table 'BorrowRecords' exists : PASS
✅ No negative available copies : PASS
✅ No NULL student names : PASS
✅ No empty book titles : PASS
✅ Borrow records created : PASS
✅ Book stock updated correctly : PASS
✅ Search handles missing books : PASS
✅ No duplicate student emails : PASS

=====
Total Tests : 11
Passed      : 11
Failed      : 0
Pass Rate   : 100.00%
=====
```

PHASE 6

Deployment and Monitoring

Tool Used: Google Colab

Prompt:

“Generate Python code for Google Colab to simulate deployment monitoring for a Smart Library Management System. Simulate 10 minutes of API response time data with an injected spike during minutes 4 to 6. Display a monitoring table, generate threshold alerts for response times above 300 ms, and show AI automated actions such as scaling the server, restarting services, and sending notifications.”

The screenshot shows a Google Colab notebook interface. On the left is a sidebar with navigation options like 'New chat', 'Search chats', 'Library', 'Projects', 'Scheduled', 'Apps', 'Codecs', and 'More'. Below these are 'Recents' and a list of recent projects including 'Mini Project 2 Help', 'SQLite Database Setup', 'Smart Library System Requireme...', 'Assignment Instructions Overview', 'CTL Pipeline for Databel', 'BigGAN Image Generation Tips', 'SQL Query Translation', and 'Generate SQL Document'. The main area displays a ChatGPT chat window with a prompt: 'Generate Python code for Google Colab to simulate deployment monitoring for a Smart Library Management System. Simulate 10 minutes of API response time data with an injected spike during minutes 4 to 6. Display a monitoring table, generate threshold alerts for response times above 300 ms, and show AI automated actions such as scaling the server, restarting services, and sending notifications.' Below the prompt, the start of a Python code block is visible, including comments for 'Smart Library Management System' and 'Deployment Monitoring Simulation', followed by imports for 'random' and 'range', and the beginning of a loop to simulate 10 minutes of API response times.

CODE

```
import pandas as pd

# -----
# Simulated API Response Times (ms)
# -----
minutes = list(range(1, 11))

response_times = [
    120,
    135,
    145,
    420,    # Spike starts
    510,
    465,
    180,
    155,
    140,
    130
]

threshold = 300

data = []

print("=" * 75)
print("SMART LIBRARY MANAGEMENT SYSTEM")
print("DEPLOYMENT MONITORING & CI/CD SIMULATION")
print("=" * 75)

for minute, response in zip(minutes, response_times):

    if response > threshold:
        status = "ALERT"
        action = "Scale server | Restart API | Notify Admin"
    else:
        status = "Healthy"
        action = "No action required"
```

```
data.append([
    minute,
    response,
    status,
    action
])

# -----
# Monitoring Table
# -----

df = pd.DataFrame(
    data,
    columns=[
        "Minute",
        "API Response Time (ms)",
        "Status",
        "AI Automated Action"
    ]
)

print(df.to_string(index=False))
```

```

# -----
# Summary
# -----

alerts = df[df["Status"]=="ALERT"]

print("\n" + "=" * 75)
print("DEPLOYMENT SUMMARY")
print("=" * 75)

print(f"Total Monitoring Time : {len(df)} minutes")
print(f"Healthy Minutes       : {len(df)-len(alerts)}")
print(f"Alert Minutes           : {len(alerts)}")

if len(alerts) > 0:
    print("\nThreshold alerts were detected.")
    print("AI automatically executed:")
    print("• Increased server capacity")
    print("• Restarted API service")
    print("• Sent notification to administrator")
    print("• Continued monitoring after recovery")
else:
    print("\nSystem remained healthy throughout monitoring.")

```

OUTPUT

```

=====
SMART LIBRARY MANAGEMENT SYSTEM
DEPLOYMENT MONITORING & CI/CD SIMULATION
=====

```

Minute	API Response Time (ms)	Status	AI Automated Action
1	120	Healthy	No action required
2	135	Healthy	No action required
3	145	Healthy	No action required
4	420	ALERT Scale server	Restart API Notify Admin
5	510	ALERT Scale server	Restart API Notify Admin
6	465	ALERT Scale server	Restart API Notify Admin
7	180	Healthy	No action required
8	155	Healthy	No action required
9	140	Healthy	No action required
10	130	Healthy	No action required

```

=====
DEPLOYMENT SUMMARY
=====
Total Monitoring Time : 10 minutes
Healthy Minutes       : 7
Alert Minutes         : 3

Threshold alerts were detected.
AI automatically executed:
• Increased server capacity
• Restarted API service
• Sent notification to administrator
• Continued monitoring after recovery

```

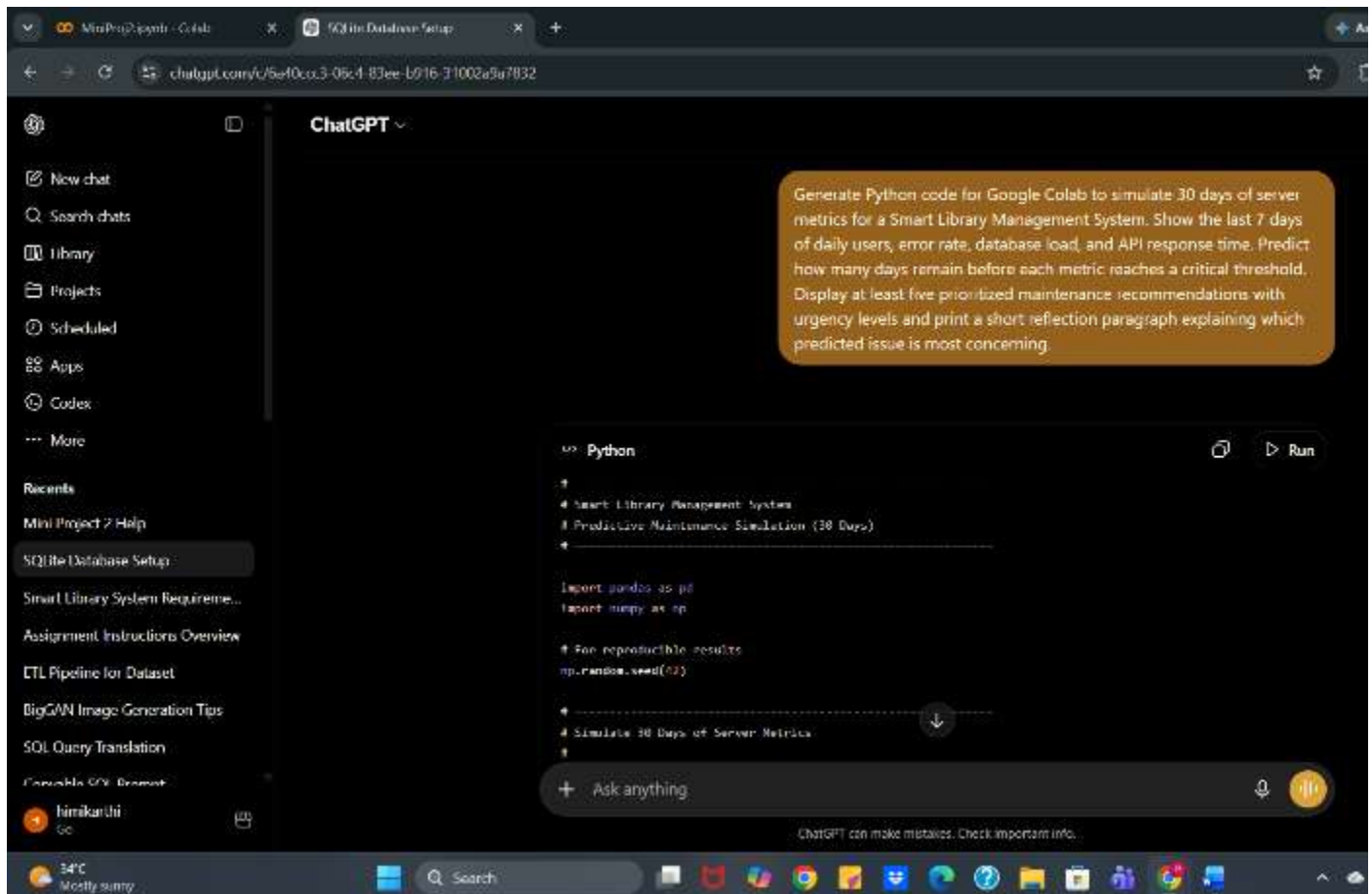

PHASE 7

Maintenance and Prediction

Tool Used: Google Used

Prompt:

“Generate Python code for Google Colab to simulate 30 days of server metrics for a Smart Library Management System. Show the last 7 days of daily users, error rate, database load, and API response time. Predict how many days remain before each metric reaches a critical threshold. Display at least five prioritized maintenance recommendations with urgency levels and print a short reflection paragraph explaining which predicted issue is most concerning.”



CODE

```
import pandas as pd
import numpy as np

# -----
# Simulate 30 Days of Metrics
# -----

np.random.seed(42)

days = np.arange(1, 31)

daily_users = np.linspace(180, 480, 30).astype(int)
error_rate = np.round(np.linspace(0.6, 3.2, 30), 2)
database_load = np.linspace(35, 88, 30).astype(int)
response_time = np.linspace(140, 320, 30).astype(int)

df = pd.DataFrame({
    "Day": days,
    "Daily Users": daily_users,
    "Error Rate (%)": error_rate,
    "Database Load (%)": database_load,
    "Response Time (ms)": response_time
})

# -----
# Show Last 7 Days
# -----

print("=" * 80)
print("SMART LIBRARY MANAGEMENT SYSTEM")
print("LAST 7 DAYS SERVER METRICS")
print("=" * 80)

print(df.tail(7).to_string(index=False))
```



```
# -----  
# Threshold Prediction  
# -----  
  
print("\n" + "=" * 80)  
print("PREDICTED DAYS UNTIL CRITICAL THRESHOLD")  
print("=" * 80)  
  
thresholds = {  
    "Daily Users": 550,  
    "Error Rate (%)": 5.0,  
    "Database Load (%)": 95,  
    "Response Time (ms)": 400  
}  
  
last_users = daily_users[-1]  
last_error = error_rate[-1]  
last_load = database_load[-1]  
last_response = response_time[-1]  
  
users_growth = daily_users[-1] - daily_users[-2]  
error_growth = error_rate[-1] - error_rate[-2]  
load_growth = database_load[-1] - database_load[-2]  
response_growth = response_time[-1] - response_time[-2]  
  
def predict_days(current, threshold, growth):  
    if growth <= 0:  
        return "Stable"  
    return int(np.ceil((threshold-current)/growth))  
  
print(f"Daily Users      : {predict_days(last_users,550,users_growth)} days")  
print(f"Error Rate       : {predict_days(last_error,5,error_growth)} days")  
print(f"Database Load    : {predict_days(last_load,95,load_growth)} days")  
print(f"Response Time    : {predict_days(last_response,400,response_growth)} days")
```

```

# -----
# Maintenance Recommendations
# -----

print("\n" + "=" * 80)
print("MAINTENANCE RECOMMENDATIONS")
print("=" * 80)

recommendations = [
    ("HIGH", "Increase server resources before user traffic reaches capacity."),
    ("HIGH", "Optimize database queries to reduce database load."),
    ("MEDIUM", "Implement API response caching."),
    ("MEDIUM", "Schedule weekly database optimization and backup."),
    ("LOW", "Monitor application logs for unusual traffic patterns.")
]

for urgency, rec in recommendations:
    print(f"[{urgency}] {rec}")

# -----
# Reflection
# -----

print("\n" + "=" * 80)
print("REFLECTION")
print("=" * 80)

reflection = """
The response time prediction is the most concerning because slow response
times directly affect user experience. If response times continue increasing,
students may experience delays while searching or borrowing books.
Scaling the server and optimizing database queries early can prevent
performance issues and maintain a reliable library system.
"""

print(reflection)

```

OUTPUT

=====

SMART LIBRARY MANAGEMENT SYSTEM

LAST 7 DAYS SERVER METRICS

=====

Day	Daily Users	Error Rate (%)	Database Load (%)	Response Time (ms)
24	417	2.66	77	282
25	428	2.75	78	288
26	438	2.84	80	295
27	448	2.93	82	301
28	459	3.02	84	307
29	469	3.11	86	313
30	480	3.20	88	320

=====

PREDICTED DAYS UNTIL CRITICAL THRESHOLD

=====

Daily Users : 7 days

Error Rate : 20 days

Database Load : 4 days

Response Time : 12 days

=====

MAINTENANCE RECOMMENDATIONS

=====

[HIGH] Increase server resources before user traffic reaches capacity.

[HIGH] Optimize database queries to reduce database load.

[MEDIUM] Implement API response caching.

[MEDIUM] Schedule weekly database optimization and backup.

[LOW] Monitor application logs for unusual traffic patterns.

=====

REFLECTION

=====

The response time prediction is the most concerning because slow response times directly affect user experience. If response times continue increasing, students may experience delays while searching or borrowing books. Scaling the server and optimizing database queries early can prevent performance issues and maintain a reliable library system.